Department of Computer Science & Engineering
Chitkara University Institute of Engineering & Technology
(Accredited by NAAC with Grade 'A+')

Practical File

Cloud Computing (23CS009)

BACHELOR OF ENGINEERING

in

Computer Science and Engineering

Submitted by:
Harshita
2310990601
G-24
5th Sem
3rd Year

Submitted to:
Dr. Prabhnoor Bachhal
Assistant Professor

CHITKARA UNIVERSITY, PUNJAB
CHANDIGARH-PATIALA NATIONAL HIGHWAY
RAJPURA (PATIALA) PUNJAB-140401 (INDIA)

INDEX

Practical No.	Practical Name	Page No.
1.	Sign up for AWS, explore free-tier services, and launch EC2 instances using Ubuntu/Linux and Windows operating systems.	1 – 9
2.	Connect to an EC2 instance via EC2 Connect, SSH Client, AWS CLI, and PuTTY SSH Client using a key pair file. Access and modify Security Group inbound and outbound rules to control traffic for EC2 instances.	10 - 16
3.	Set up and configure web servers on EC2 instances, including Apache and Nginxserver. Configure auto-scaling for EC2 instances to handle variable traffic.	17 – 33
4.	Create an Amazon S3 bucket, upload objects, set up access controls with bucket policies, and use it for hosting static content.	34 - 44
5.	Set up an AWS Elastic Load Balancer (ELB) to distribute traffic among EC2 instances.	
6.	Set up Amazon VPC networking, including subnets, route tables, security groups. Scenario 1: VPC With a Public Subnet Only (Standalone Web) Scenario 2: VPC with Public and Private Subnets (3 Tier App)	
7.	Configure Amazon CloudWatch to monitor EC2 instance metrics and set up alarms.	
8.	Create an SNS, subscribe and publish a message, set up an SQS queue, integrate it with an EC2 instance, and use SES to send event notifications via email.	

Practical No. 1

Practical Title: Sign up for AWS, Explore Free-Tier Services, and Launch EC2 Instances using Ubuntu and Windows.

Objective:

1. To create an AWS account and explore free-tier services.
2. To launch an EC2 instance with Ubuntu and Windows OS.

Step 1: AWS Account Signup

1. Visit AWS Portal: Open https://aws.amazon.com/ and click on "Create an AWS Account".
2. Enter account details, set account name, click Continue.
3. Enter billing details (AWS Free Tier requires card verification).
4. Verify via OTP sent to your mobile.
5. Select Free Tier (Basic Support) plan.
6. Sign in to AWS Console after account activation.

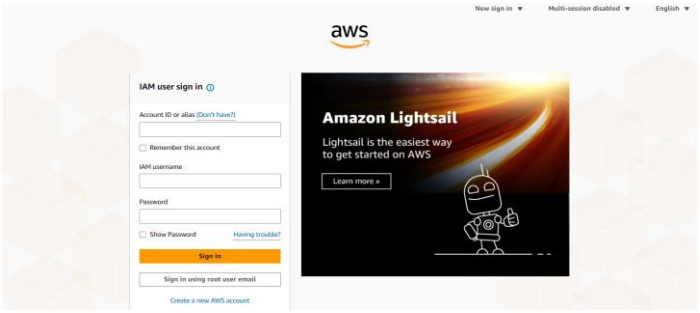

Figure 1.1

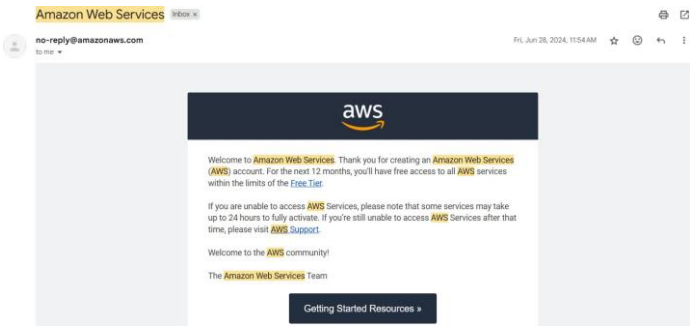

Figure 1.2

Step 2: Explore AWS Free-Tier Services

1. Navigate to AWS Console → Explore EC2, S3, Lambda, RDS, etc.
2. Check Free Tier Dashboard from Billing → Free Tier.

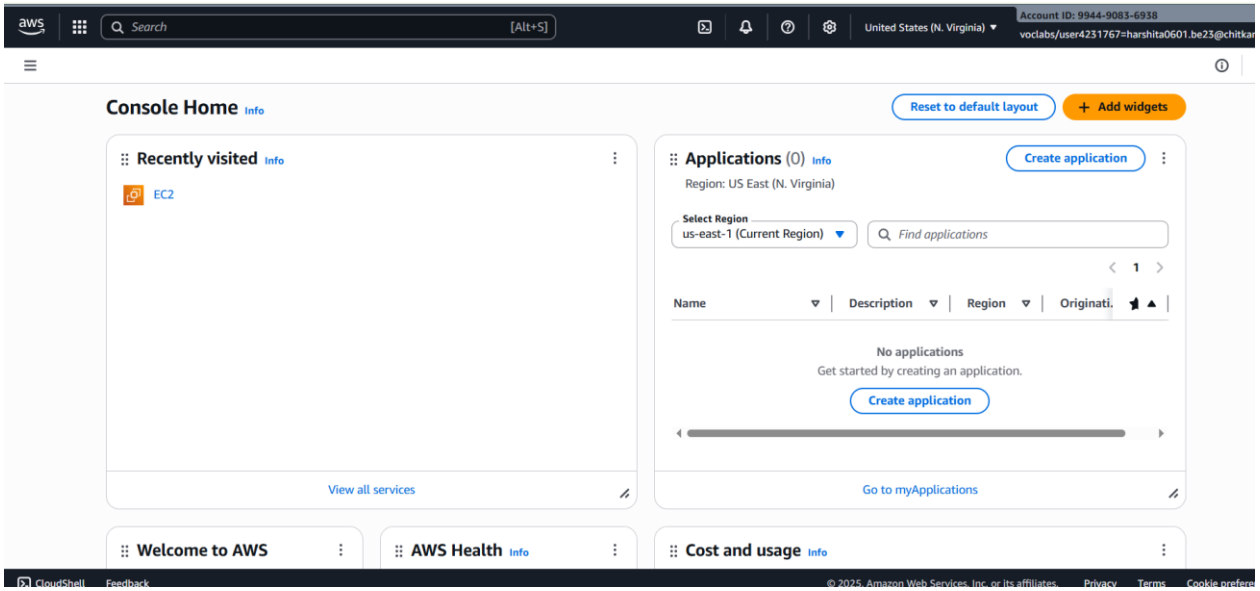

Figure 1.3

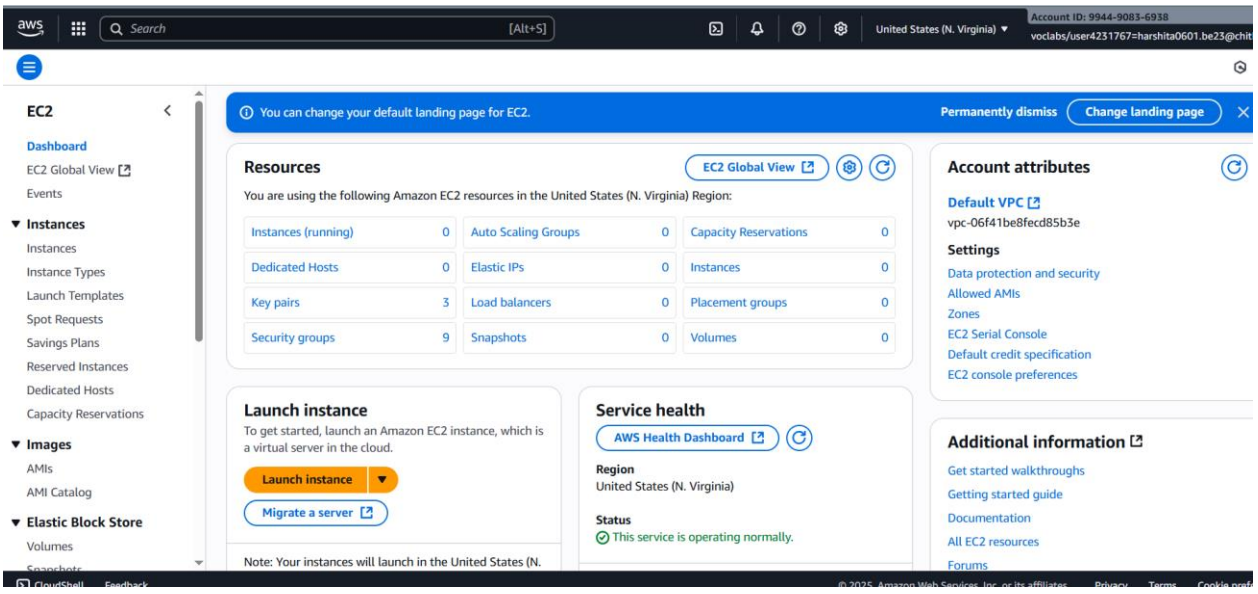

Figure 1.4

Step 3: Launch EC2 Instances

(A) Ubuntu EC2 Instance

1. Go to EC2 → Launch Instance.
2. Select AMI: Ubuntu 22.04 LTS (Free Tier).
3. Instance type: t2.micro.
4. Keep default settings, storage: 8 GB.
5. Add Name tag: Ubuntu_Instance.
6. Configure Security Group: Allow SSH (Port 22).
7. Create & download key pair (.pem).
8. Launch instance and connect via SSH:- ssh -i my-key.pem ubuntu@ec2<your-public-ip>.compute-1.amazonaws.com

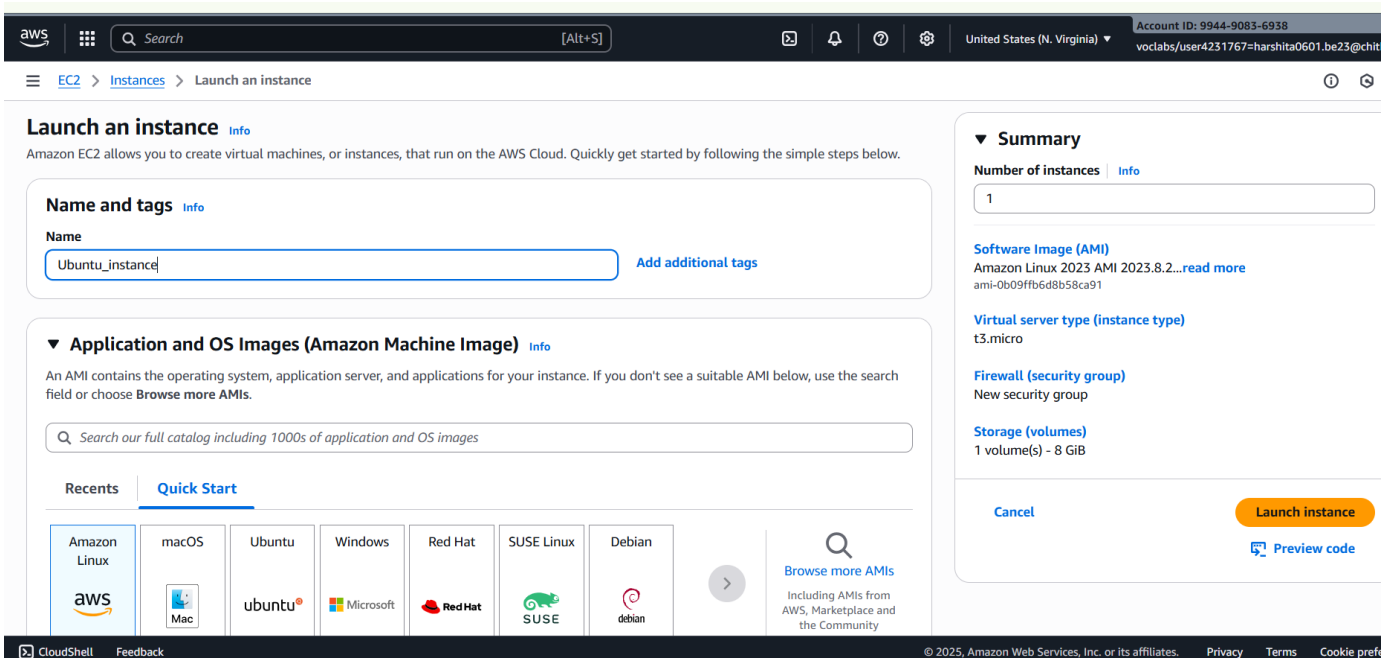

Figure 1.5

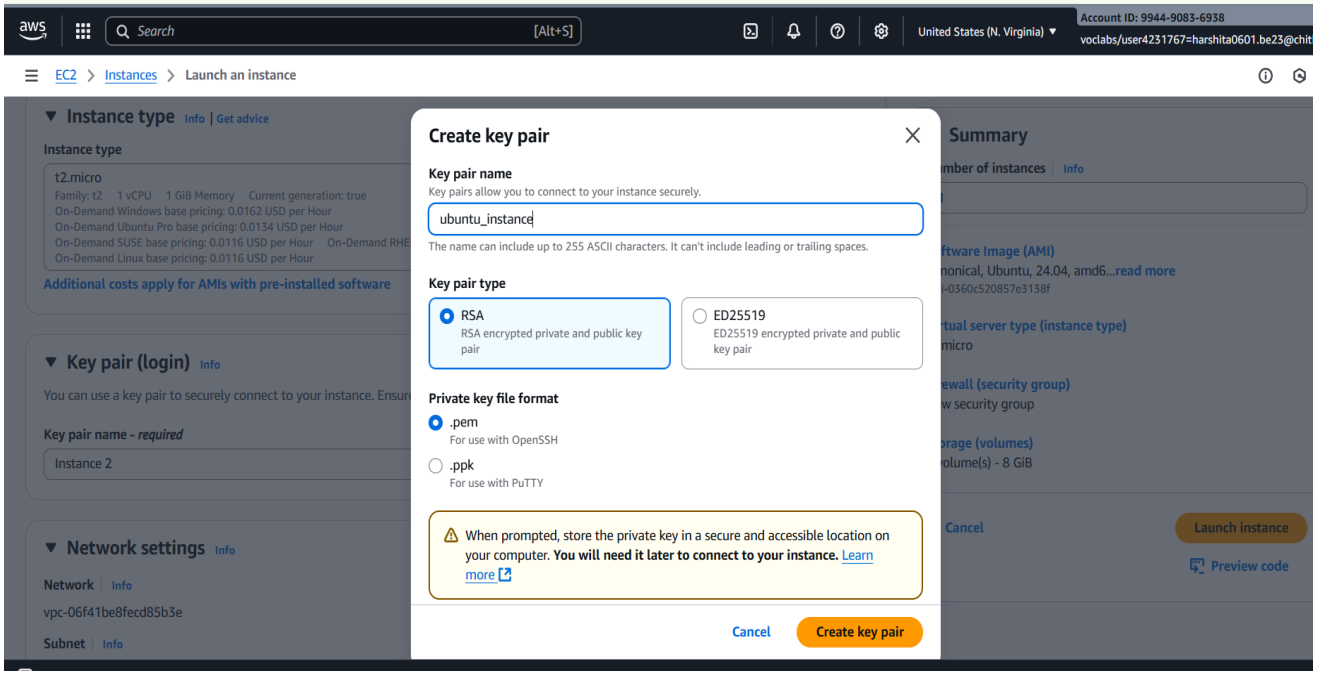

Figure 1.6

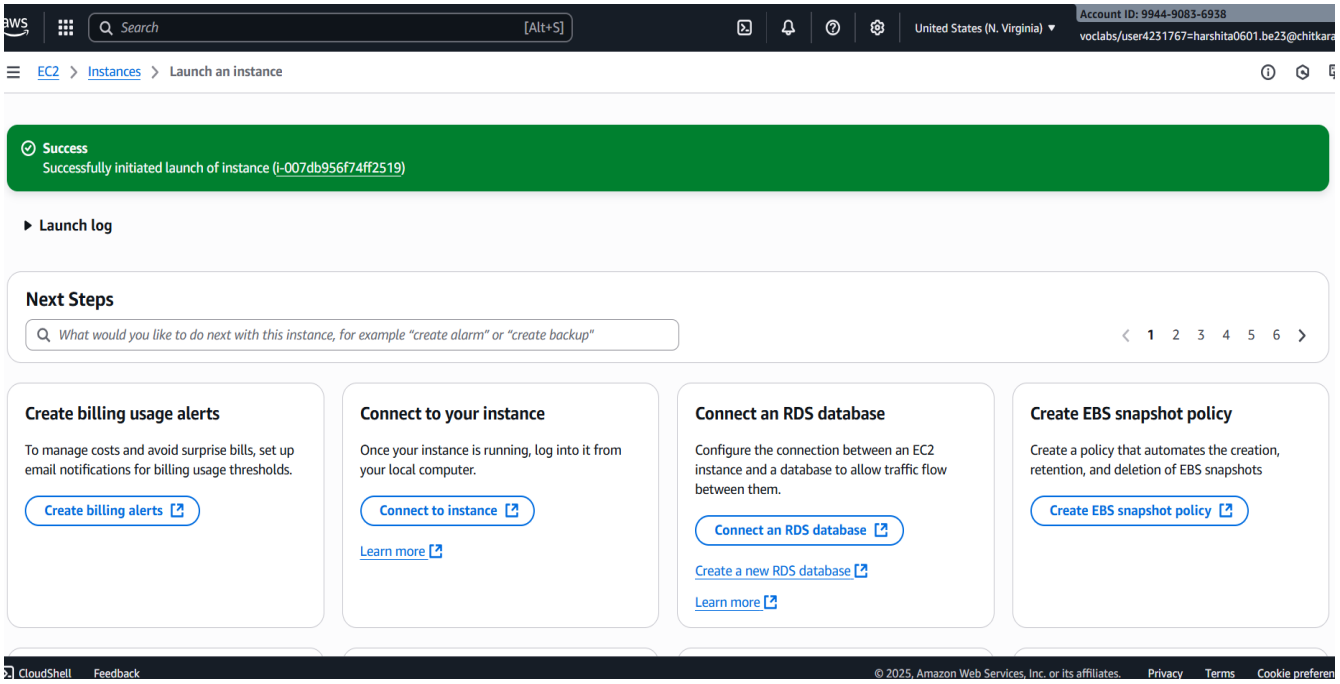

Figure 1.7

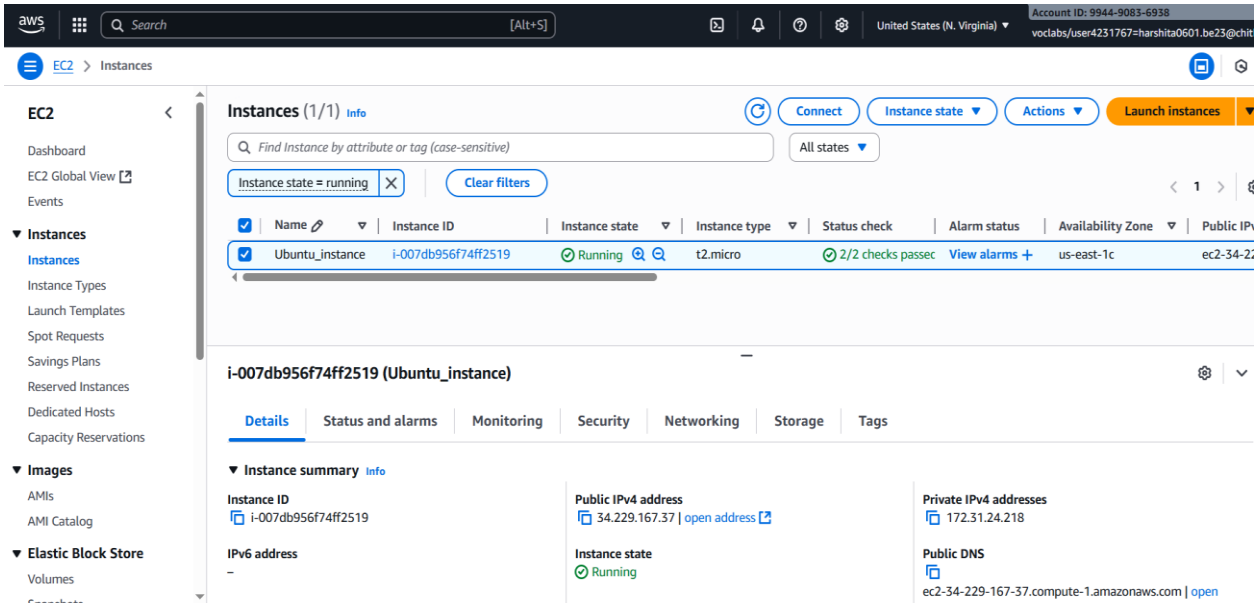

Figure 1.8

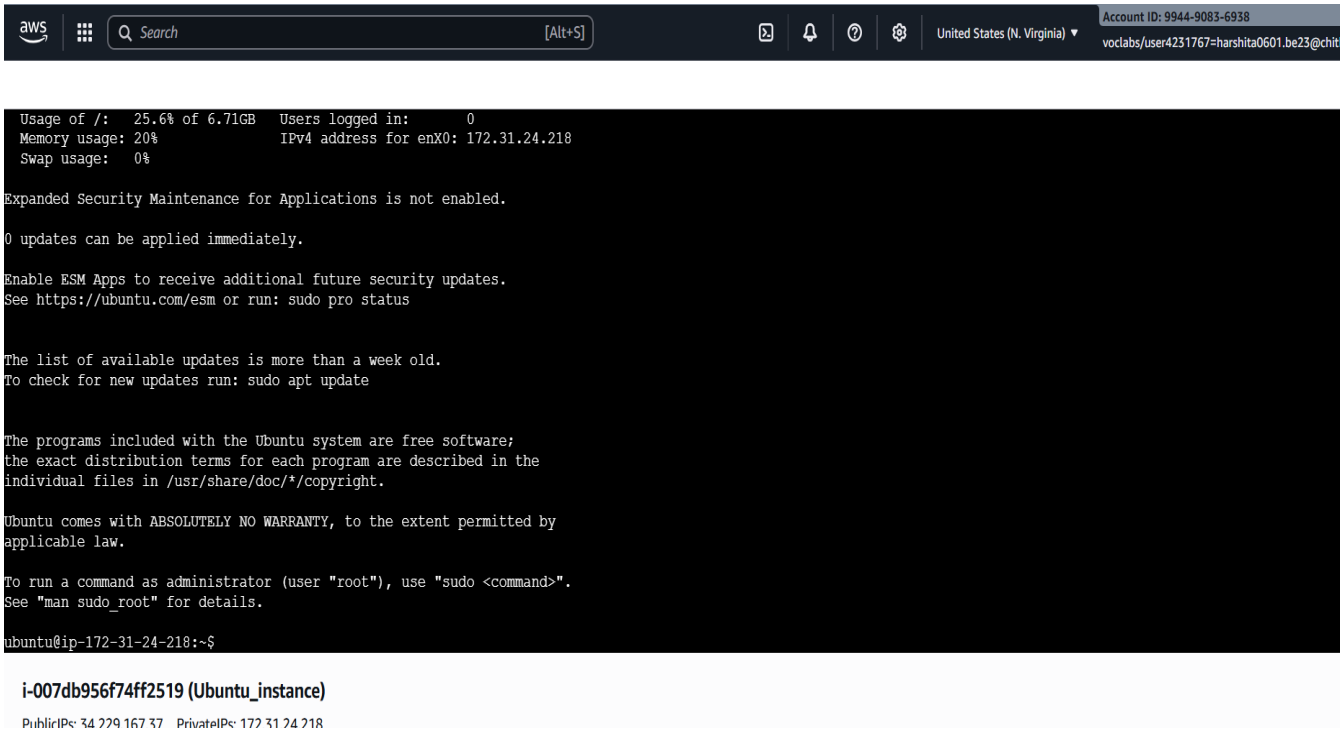

Figure 1.9

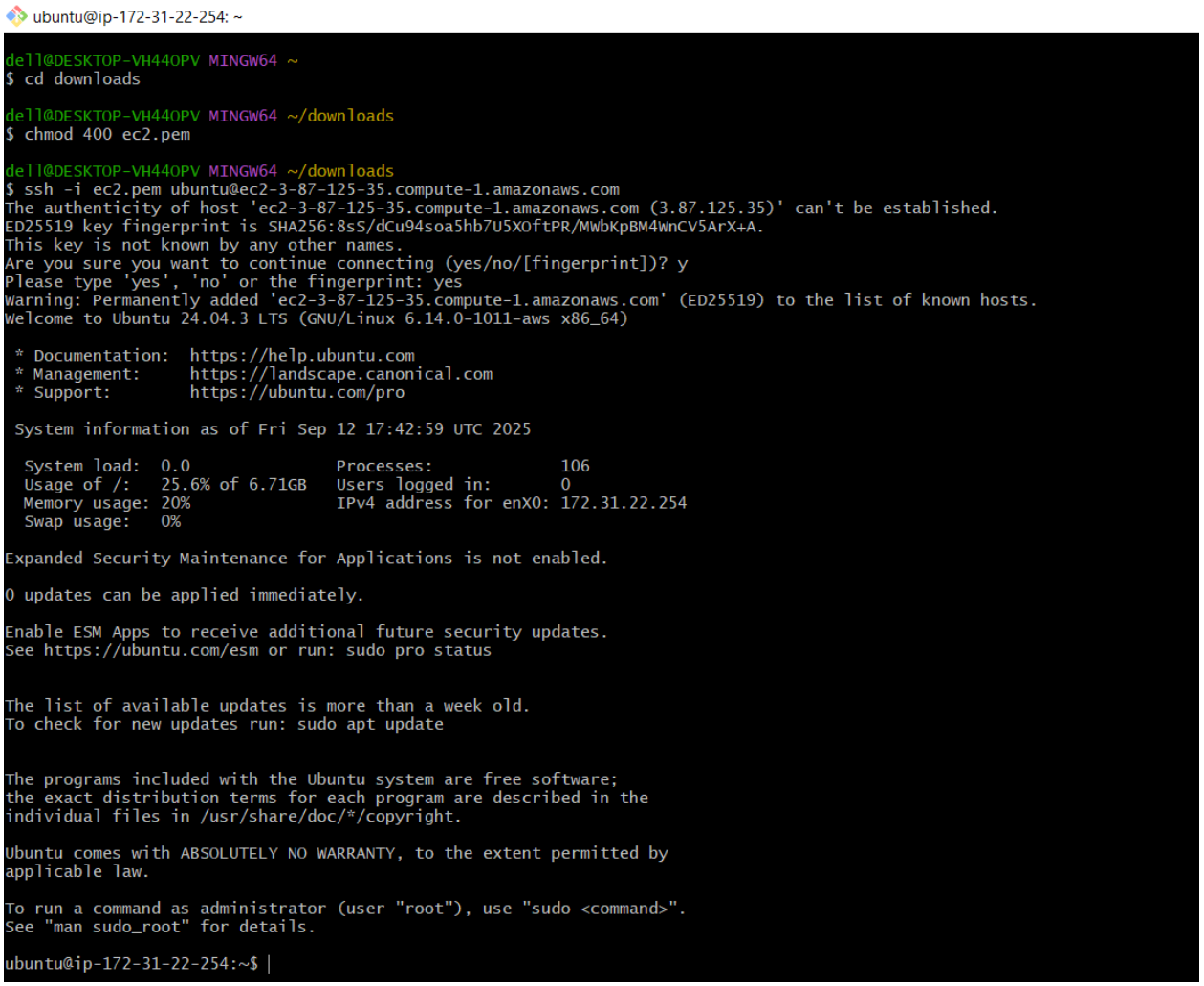
ubuntu@ip-172-31-22-254: ~
dell@DESKTOP-VH44OPV MINGW64 ~
$ cd downloads
dell@DESKTOP-VH44OPV MINGW64 ~/downloads
$ chmod 400 ec2.pem
dell@DESKTOP-VH44OPV MINGW64 ~/downloads
$ ssh -i ec2.pem ubuntu@ec2-3-87-125-35.compute-1.amazonaws.com
The authenticity of host 'ec2-3-87-125-35.compute-1.amazonaws.com (3.87.125.35)' can't be established.
ED25519 key fingerprint is SHA256:8sS/dCu94soa5hb7U5XOftPR/MWbKpBM4WnCV5ArX+A.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added 'ec2-3-87-125-35.compute-1.amazonaws.com' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1011-aws x86_64)
* Documentation: https://help.ubuntu.com
* Management: https://landscape.canonical.com
* Support: https://ubuntu.com/pro
System information as of Fri Sep 12 17:42:59 UTC 2025
System load: 0.0 Processes: 106
Usage of /: 25.6% of 6.71GB Users logged in: 0
Memory usage: 20% IPv4 address for enX0: 172.31.22.254
Swap usage: 0%
Expanded Security Maintenance for Applications is not enabled.
0 updates can be applied immediately.
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status
The list of available updates is more than a week old.
To check for new updates run: sudo apt update
The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
ubuntu@ip-172-31-22-254:~$

Figure 1.10

(B) Windows EC2 Instance

1. Repeat steps 1–3, but select Microsoft Windows Server 2025 Base.

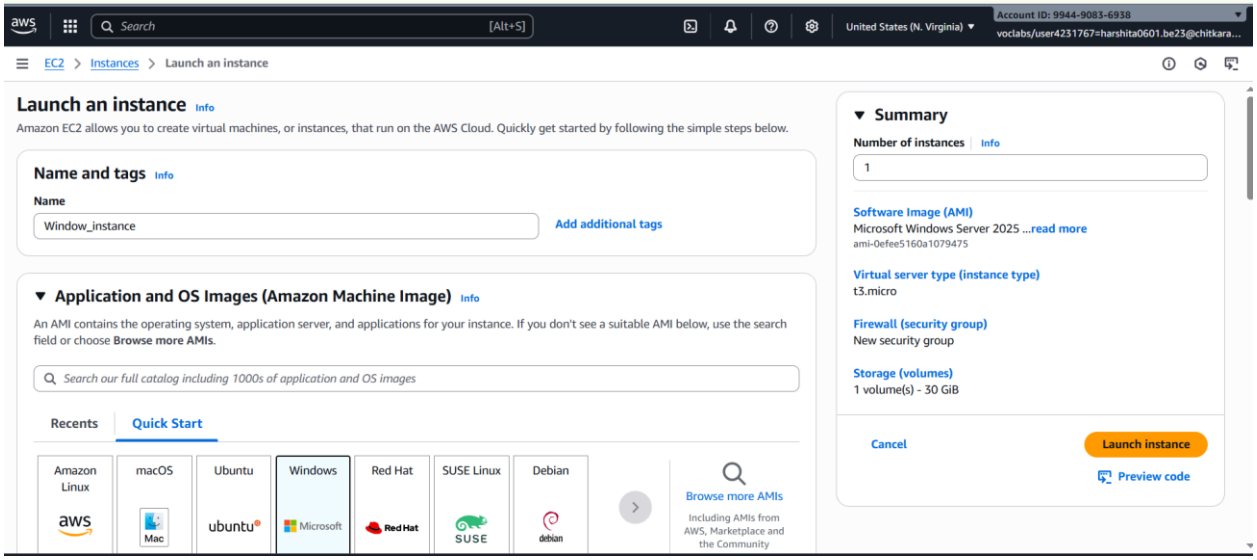

Figure 1.11

2. Allow RDP (Port 3389).
3. Download key file for password decryption.

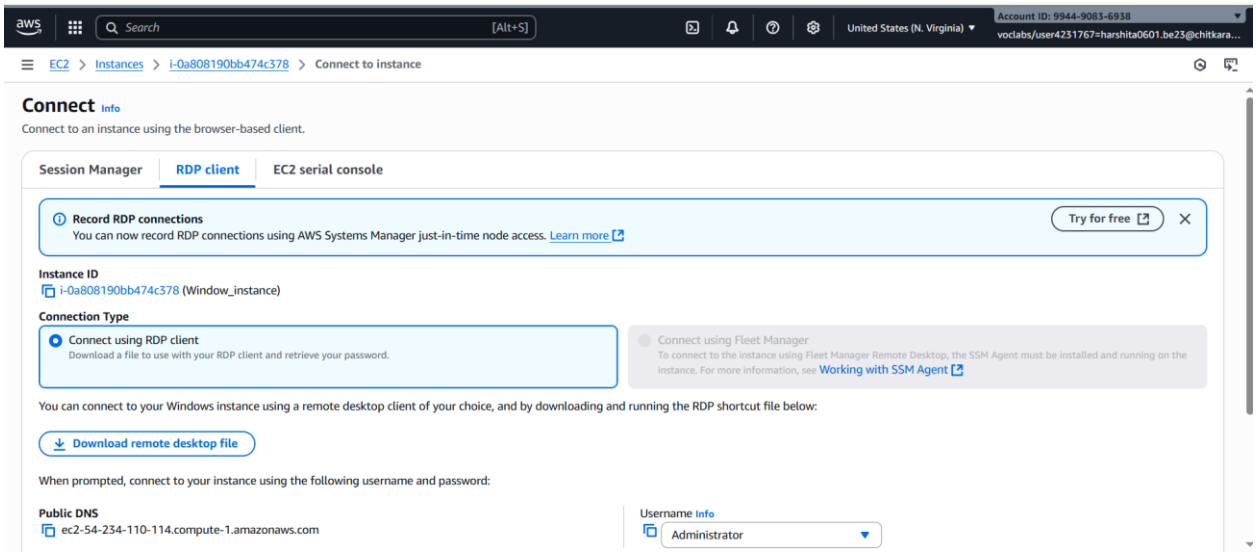

Figure 1.12

4. Launch → Get Password → Decrypt with key file.

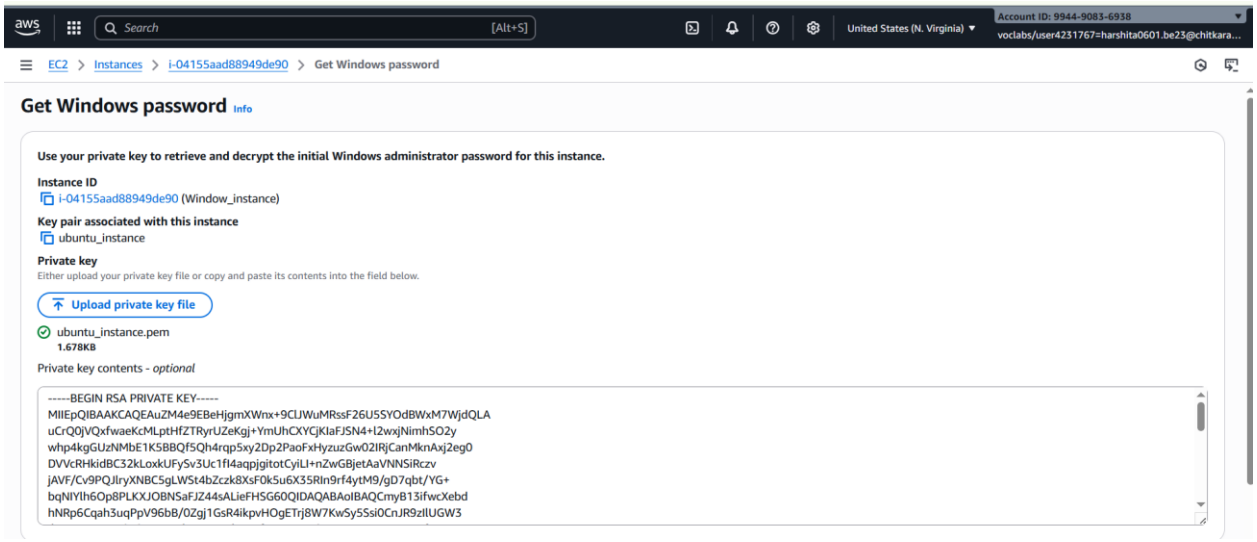

Figure 1.13

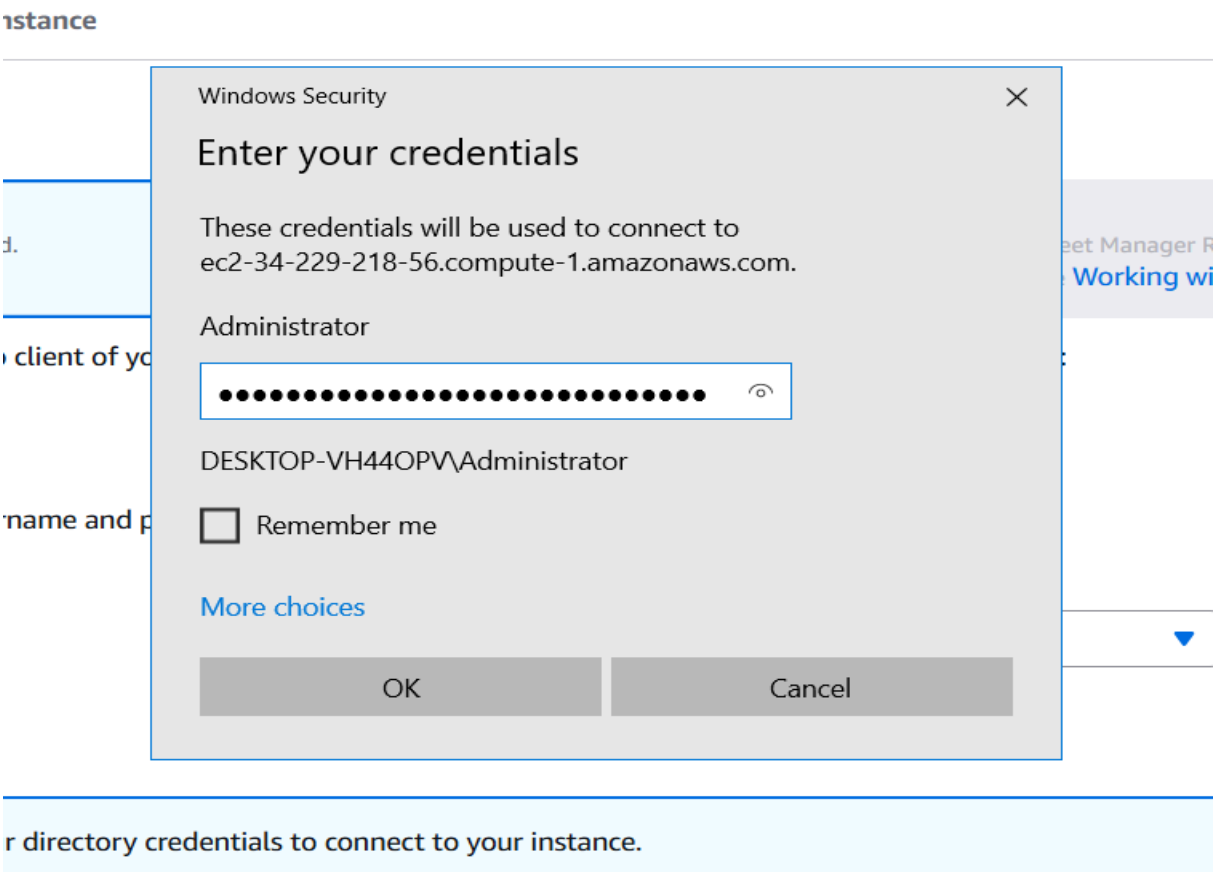

Figure 1.14

5. Connect using Remote Desktop Connection (RDP).

Figure 1.15

Learning Outcomes:

1. Successfully signed up for AWS and explored free-tier services.
2. Launched and connected to both Ubuntu and Windows EC2 instances.
3. Understood basics of creating & managing virtual machines on AWS.

Practical No. 2

Practical Title: Connect to an EC2 Instance via EC2 Connect, SSH Client, and PuTTY. Modify Security Group Rules.

Objective:

2.1 Connect to an EC2 instance using a key pair file via

A. EC2 Connect

B. SSH Client and

C. PuTTY SSH Client

2.2 Access and modify Security Group inbound and outbound rules to control traffic for EC2 instances.

2.1 Connect to an EC2 instance using a key pair

Step 1: Launch an EC2 Instance

1. Open AWS Management Console → EC2 → Launch Instance.
2. Select Ubuntu 22.04 LTS AMI (Free-Tier Eligible).
3. Choose t2.micro instance type.
4. Create a key pair (e.g., ec2-key.pem) and download it.
5. In Security Group, allow:
    - SSH (Port 22) from your IP.
    - HTTP (Port 80) for web access.

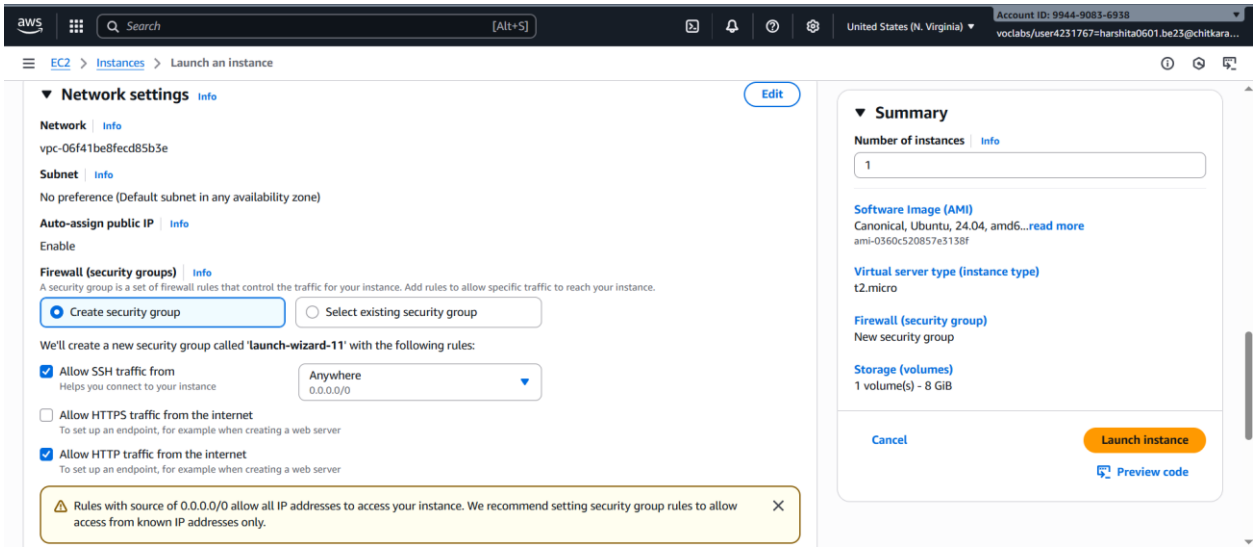

Figure 2.1

Step 2: Connect to EC2 Instance

(A) EC2 Instance Connect (Browser-Based)

1. Go to EC2 → Instances → Select Instance → Connect.
2. Choose EC2 Instance Connect tab.

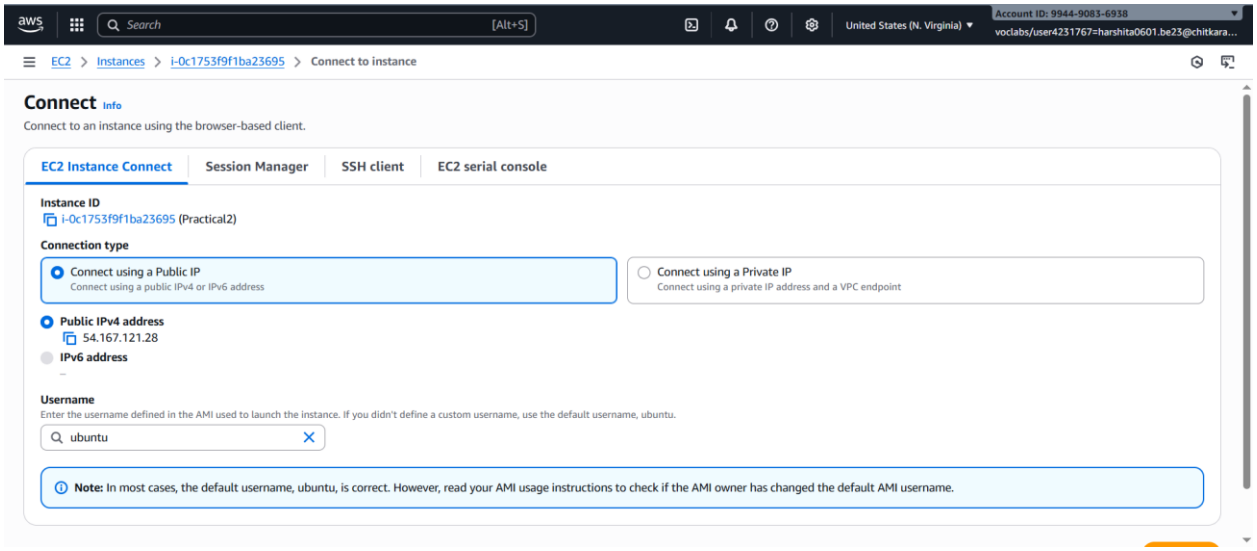

Figure 2.2

3. Click Connect → browser-based terminal opens.

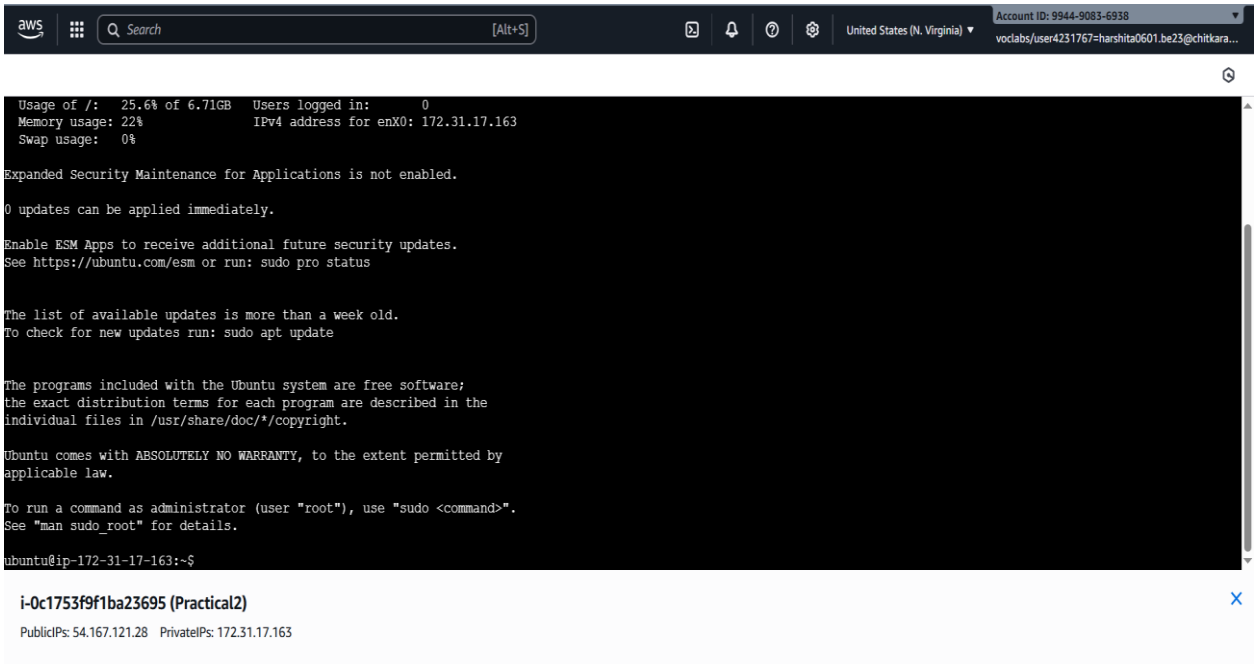

Figure 2.3

(B) SSH Client (Linux/macOS Terminal)

Connect Using SSH Client on Linux/Mac

1. Open Terminal
2. Navigate to the folder where your .pem file is saved: cd /path/to/your/file
3. Change Permissions of the Key File

   Type this command: chmod 400 my-key.pem
4. Connect to your EC2 instance.

   Type the SSH command:

   ssh -i my-key.pem ubuntu@ec2<your-public-ip>.compute-1.amazonaws.com

Replace <your-public-ip> with the Public IPv4 address of your EC2 instance.

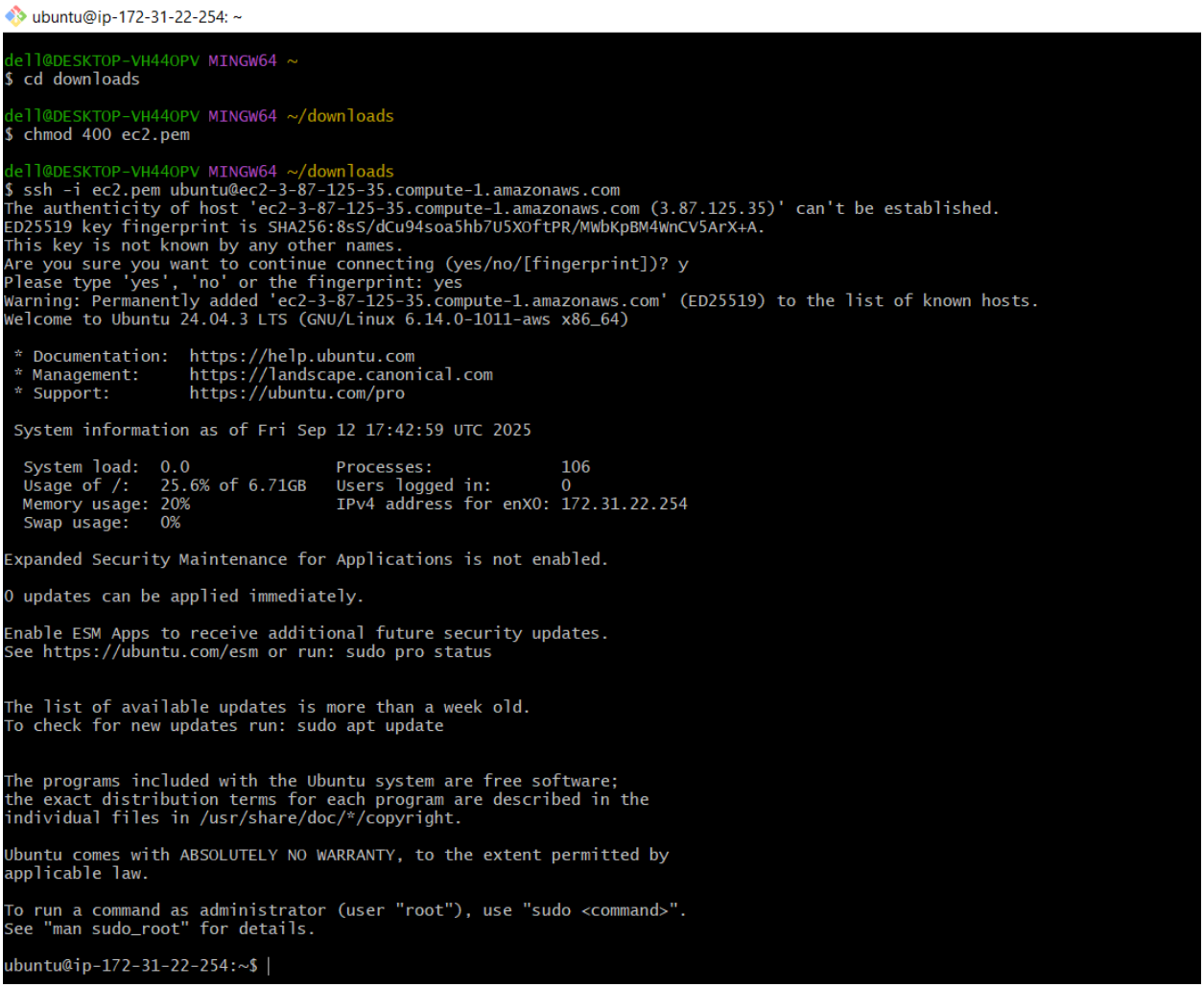

Figure 2.4

(C) PuTTY (Windows)

1. Convert .pem to .ppk using PuTTY gen.
2. Open PuTTY, enter the EC2 Public IP.
3. Under SSH → Auth, load the .ppk key file.
4. Click Open → log in as ubuntu

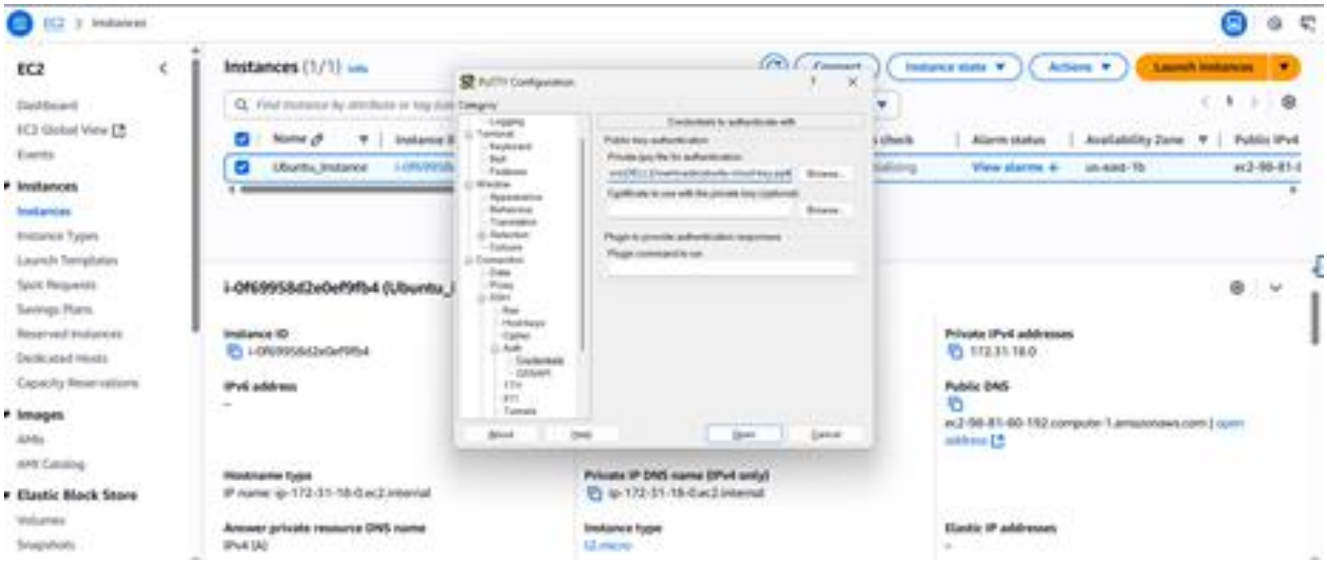

Figure 2.5

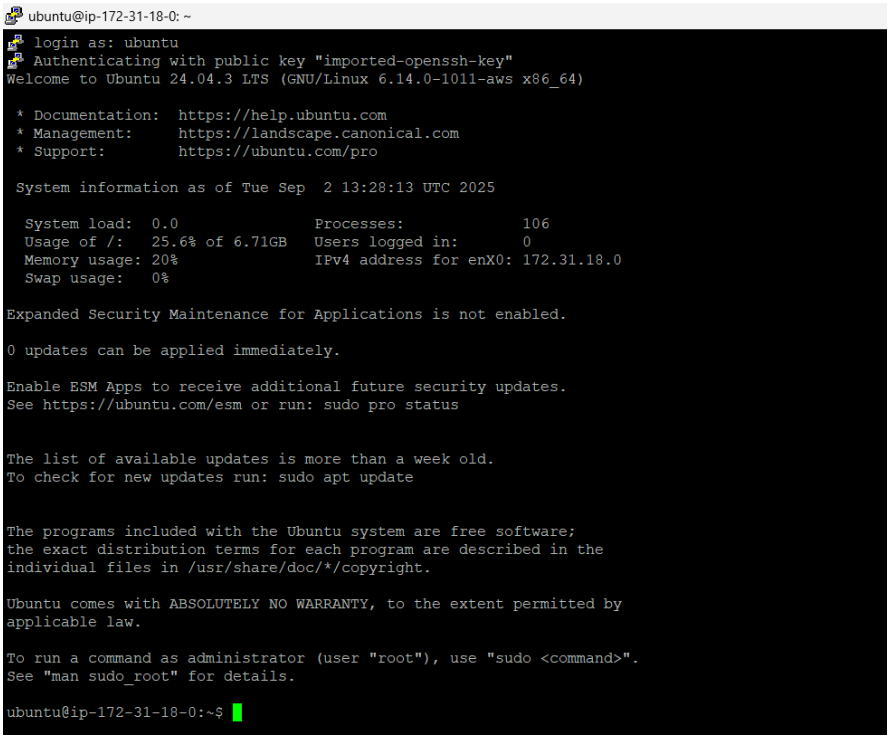

Figure 2.6

2.2 Access and modify Security Group inbound and outbound rules to control traffic for EC2 instances.

(A) Edit Inbound Rules

1. Go to EC2 → Security Groups → Select SG → Edit inbound rules.
2. Add a rule to allow HTTP (Port 80) from Anywhere (0.0.0.0/0).
3. Remove or restrict SSH access if not needed.

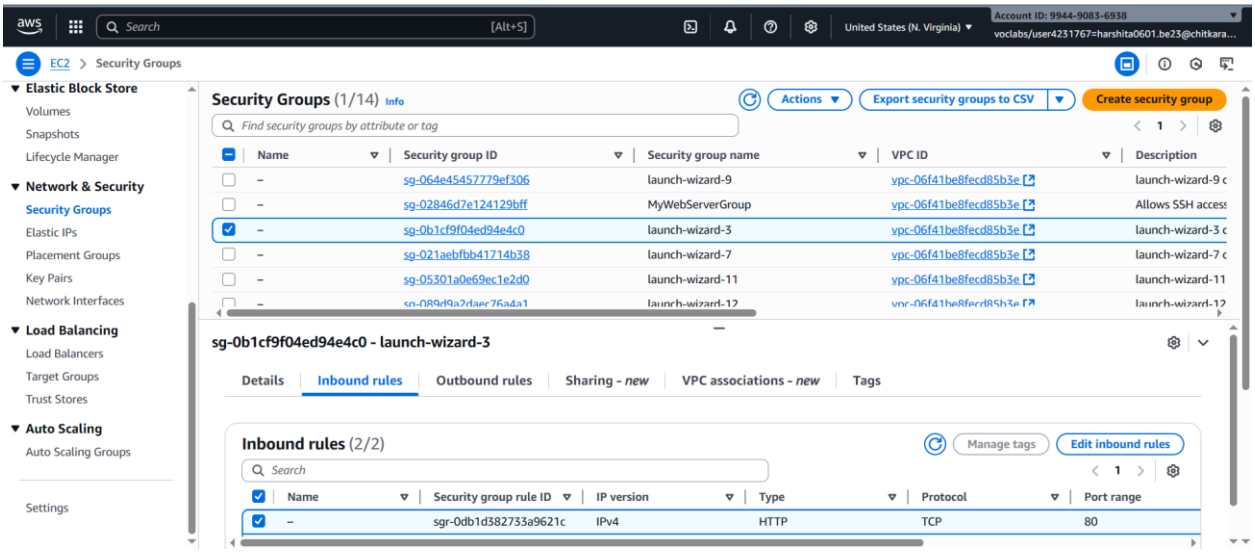

Figure 2.7

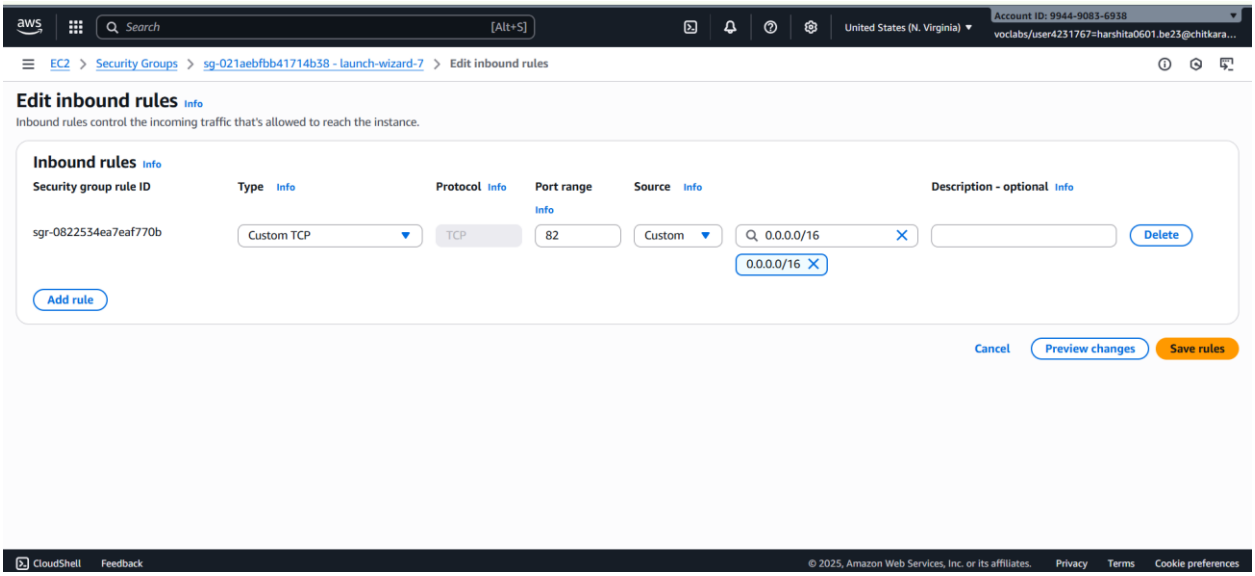

Figure 2.8

(B) Edit Outbound Rules:

1. Go to Outbound rules.
2. Configure allowed outbound traffic (default: all traffic).
3. Save changes.

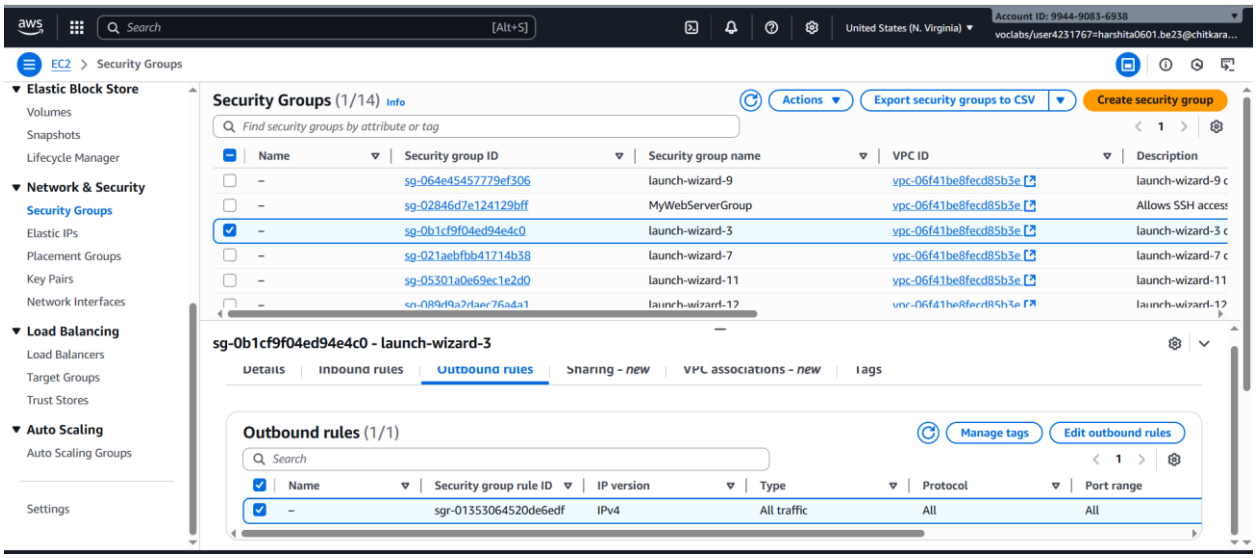

Figure 2.9

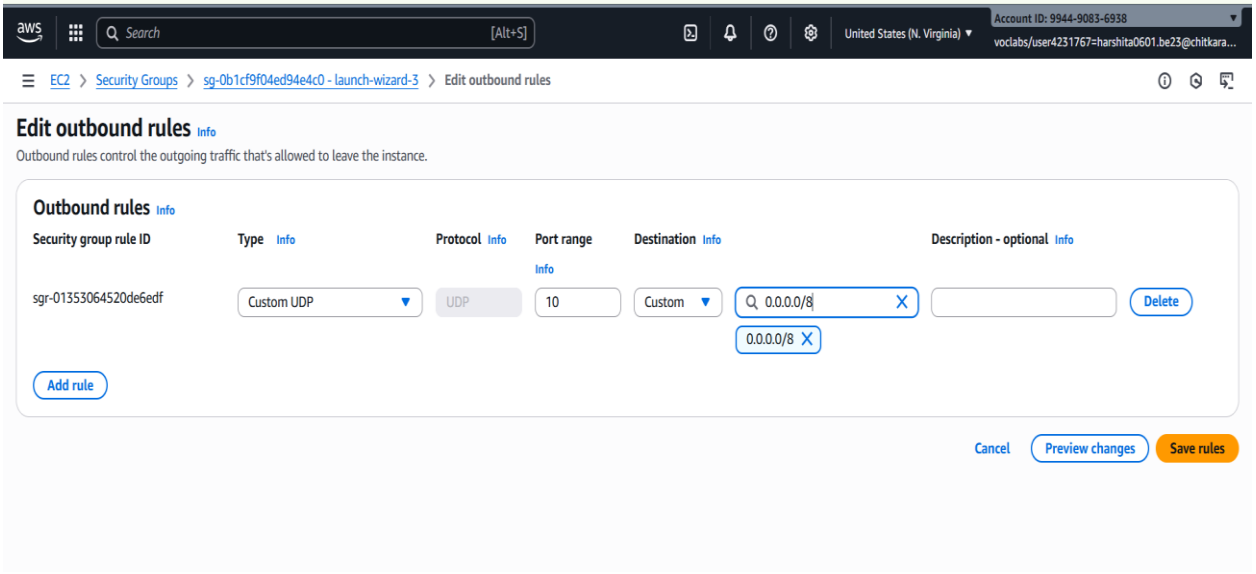

Figure 2.10

Learning Outcomes:

- Successfully connected to EC2 instances using EC2 Instance Connect, SSH Client, AWS CLI, and PuTTY.
- Implemented secure authentication using a key pair file.
- Configured and modified Security Group inbound and outbound rules to control network traffic.
- Gained an understanding of how Security Groups function as a virtual firewall for AWS resource.

Practical No. 3

Practical Title: Set up and configure web servers on EC2 instances, including Apache and Nginxserver. Configure auto-scaling for EC2 instances to handle variable traffic.

Objective:

A. To install and configure Apache and Nginx web servers on EC2 instances.

B. To configure Auto Scaling Groups (ASG) for EC2 instances to handle variable traffic automatically.

(A) Install and Configure Apache and Nginx web servers on EC2 instances

Step 1: Launch EC2 Instances

1. Open AWS Management Console → EC2 → Launch Instance
2. Choose Ubuntu 24.04 LTS (HVM)

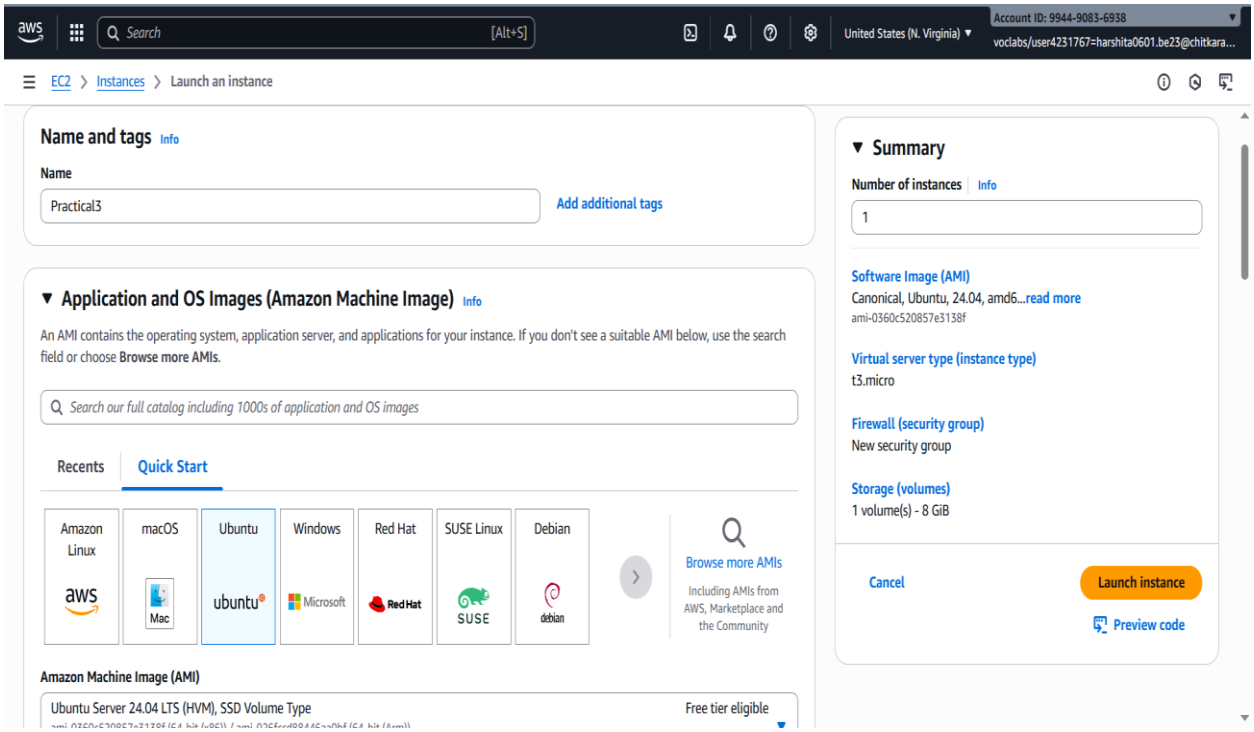

Figure 3.A.1

3. Select t3.micro instance type.
4. Create/Select a key pair (e.g. web key.pem)

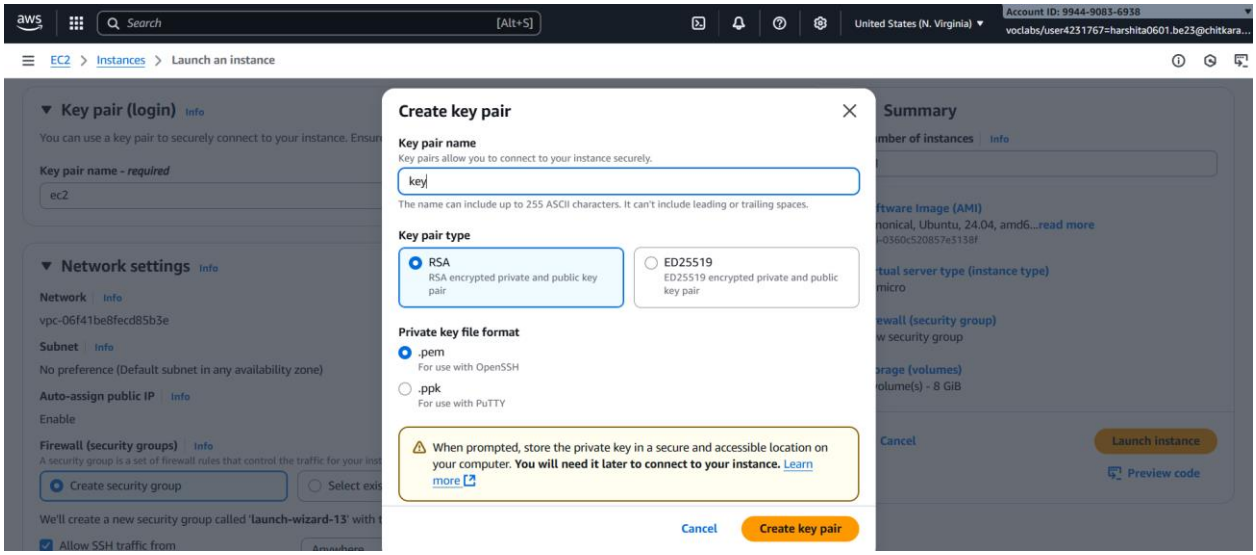

Figure 3.A.2

5. Configure Security Groups to allow:
    - HTTP (Port 80).
    - HTTPs (Port 443)
    - SSH (Port 22)

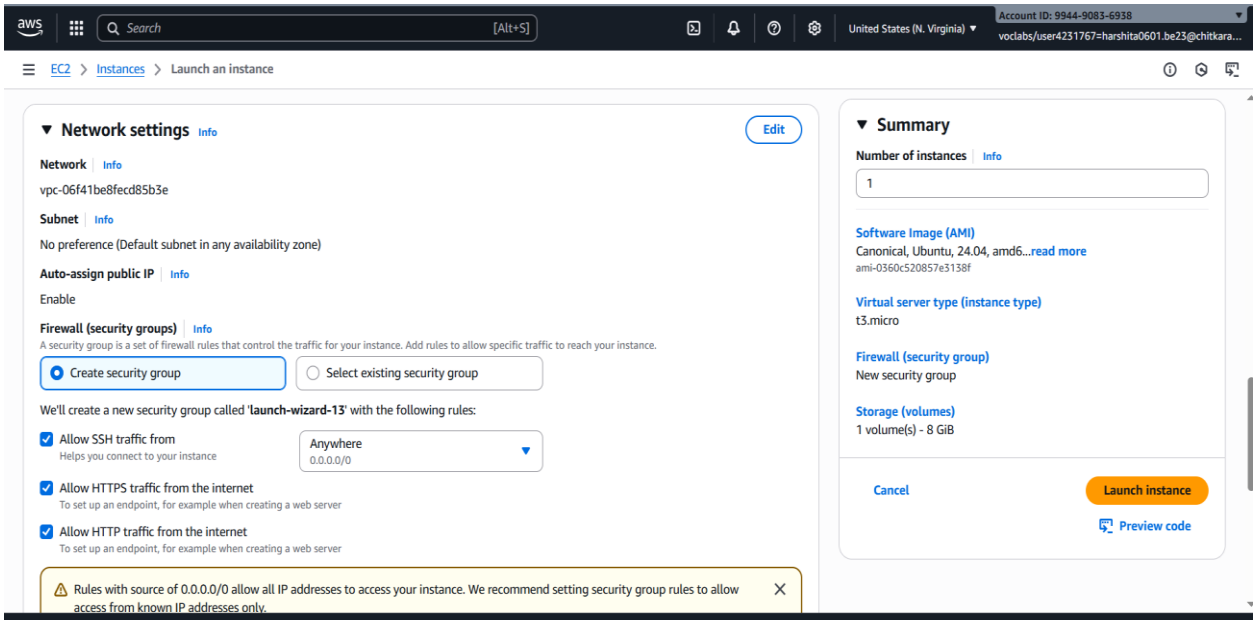

Figure 3.A.3

6. Launch the instance.

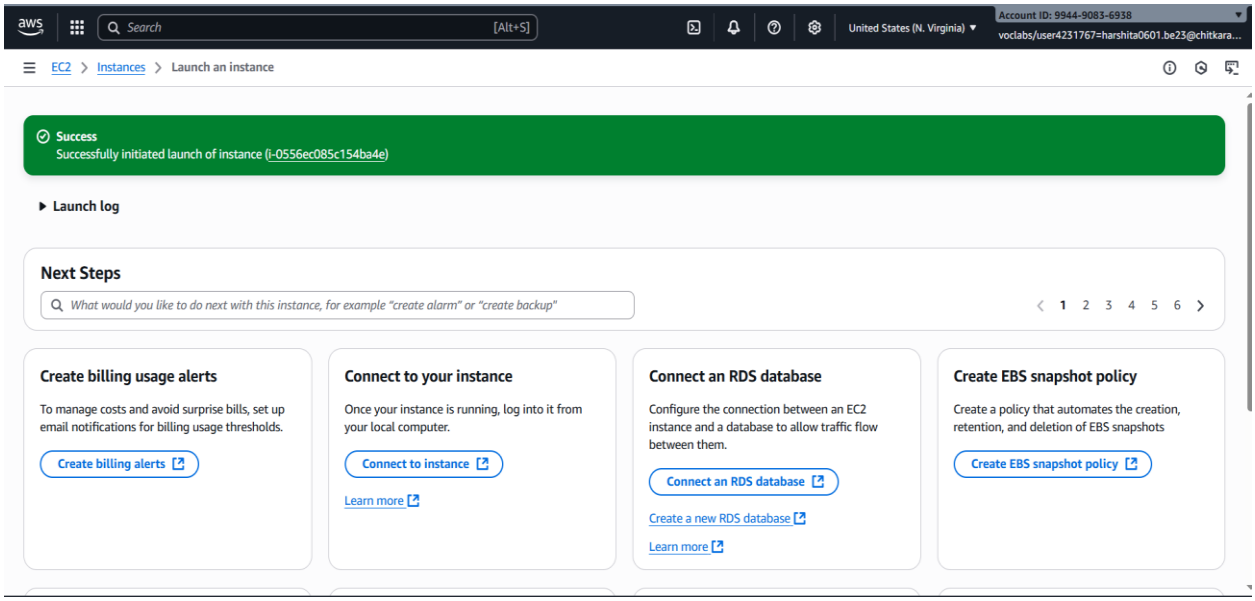

Figure 3.A.4

Step 2: Update and Upgrade the instance

- Run this first to ensure all packages are up to date: sudo apt update

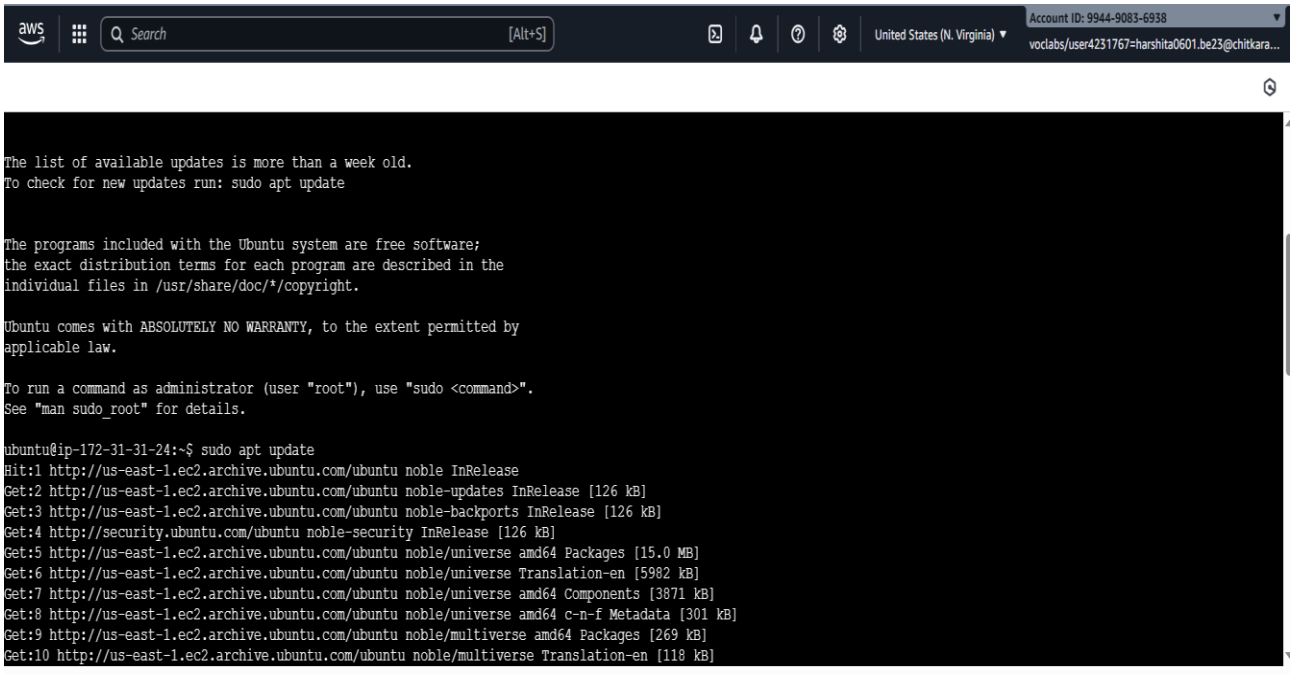

Figure 3.A.5

- Run this first to upgrades the installed packages to their latest versions.: sudo apt upgrade

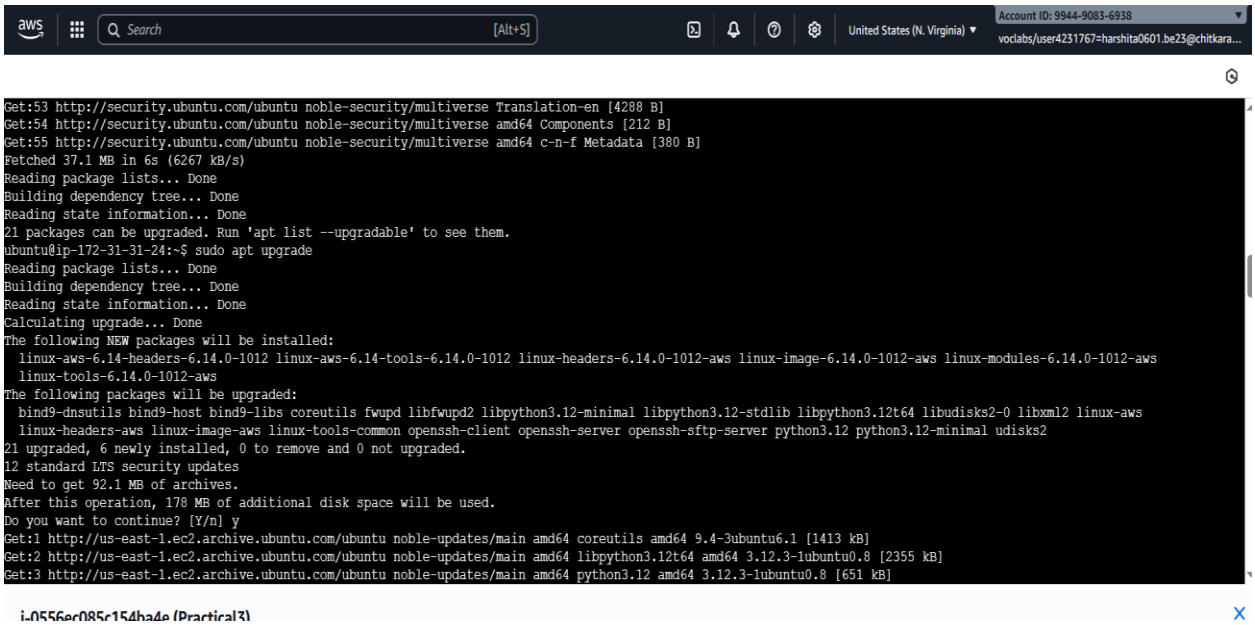

Figure 3.A.6

Step 3: Install Apache

- sudo apt install apache2 -y

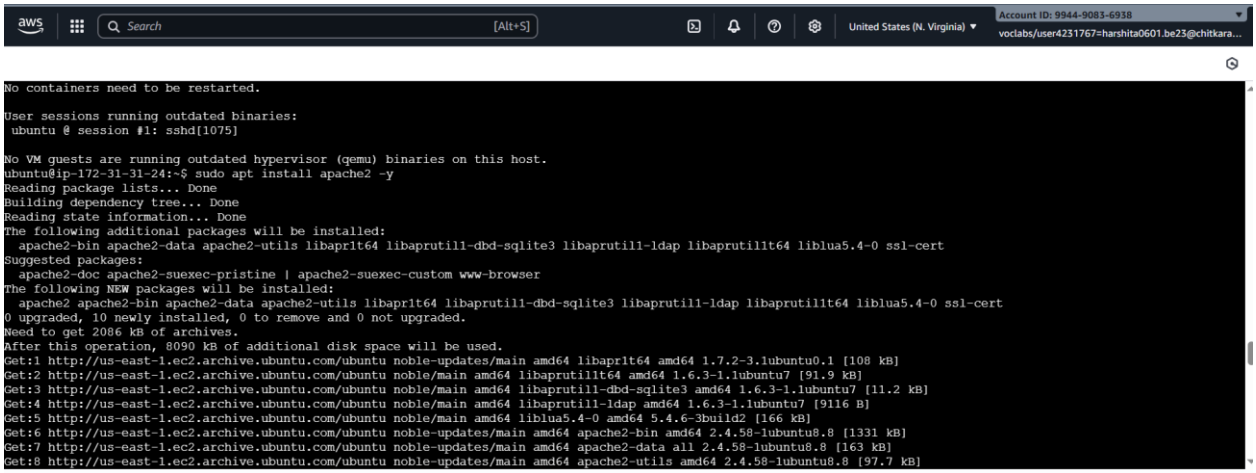

Figure 3.A.7

- sudo systemctl start apache2 → start Apache immediately
- sudo systemctl enable apache2 → make it start automatically on boot
- sudo systemctl status apache2 → verifies apache is working

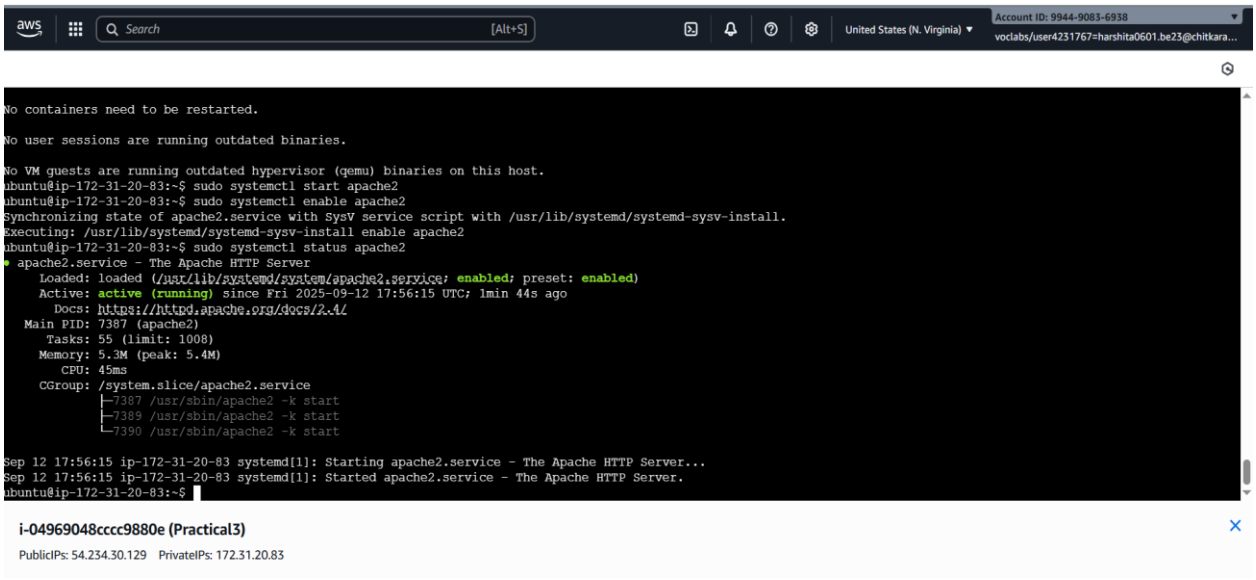

Figure 3.A.8

Step4: Test in browser:

- Copy your EC2 public IP from the AWS console.
- Open in your browser → http://<your-public-ip>/

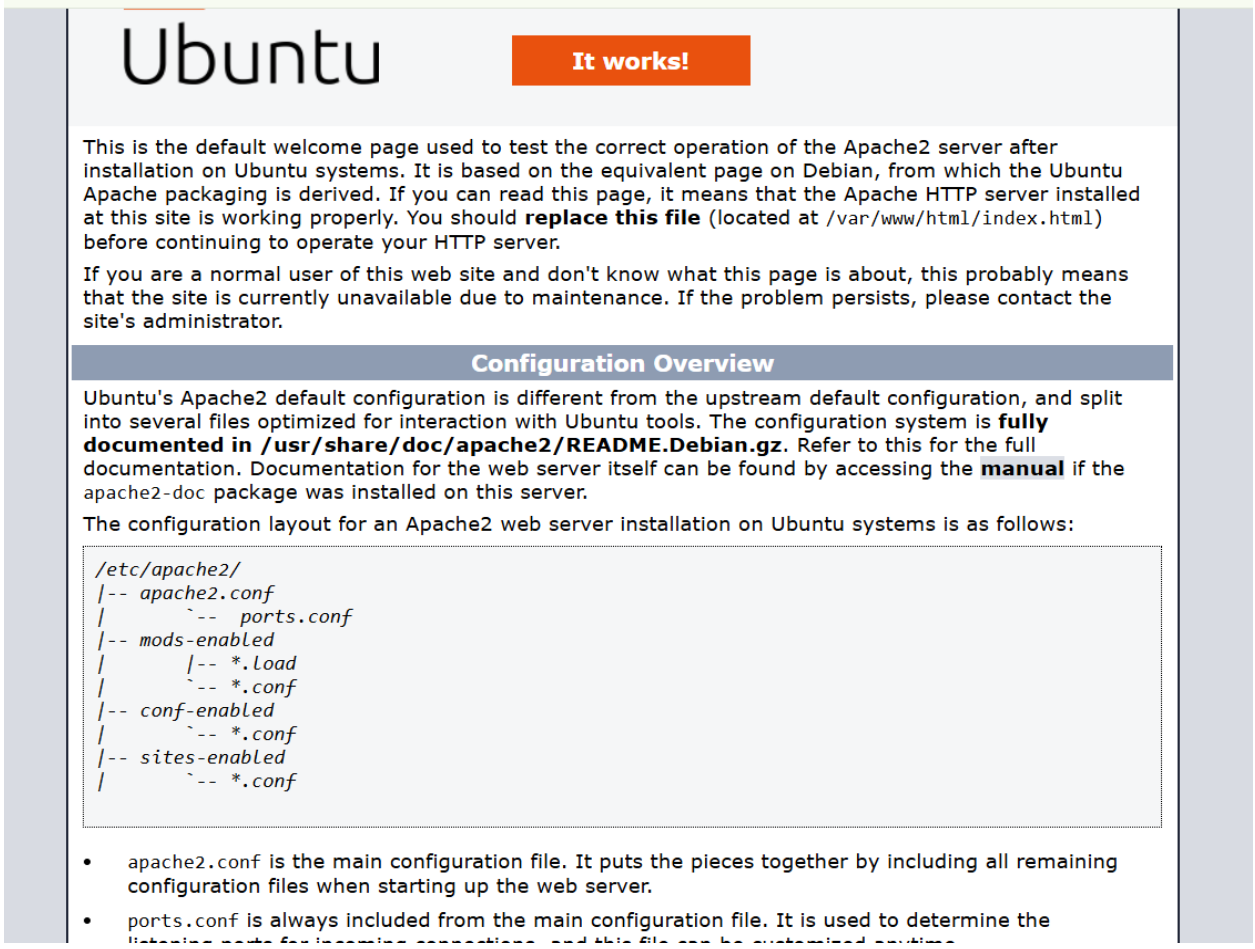

Figure 3.A.9

Step 5: Customizing Web Pages using Apache:

- Run this command in the ubuntu terminal
  1. echo '<body style="background-color:lavender; color:darkblue; font-family:sans-serif;"><h1>My Styled Apache Page</h1> This page has custom colors! </body>' | sudo tee /var/www/html/index.html

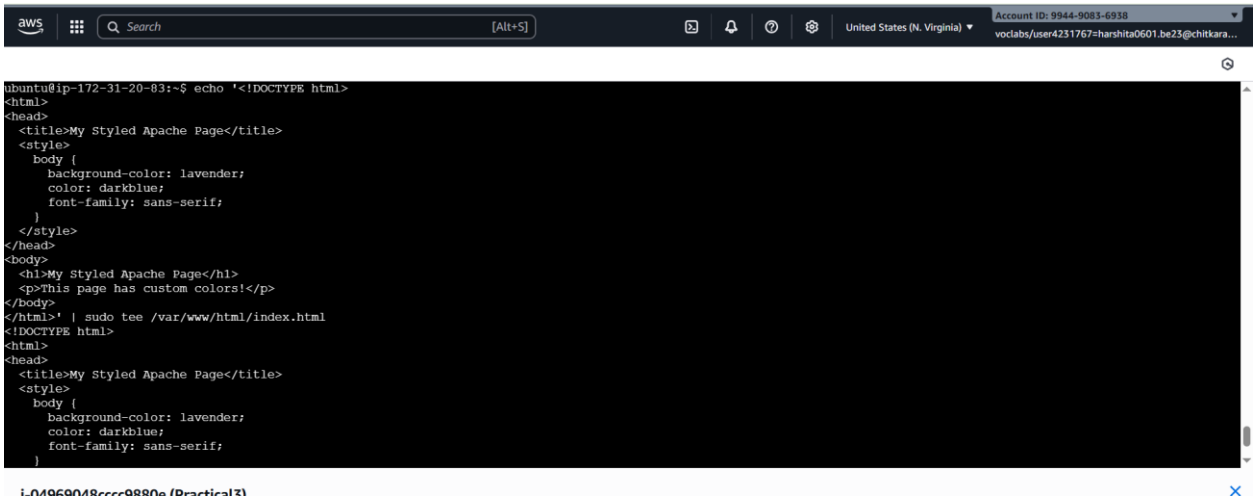

Figure 3.A.10

After running this command, refresh your browser page and you will see the changes

Figure 3.A.11

2. echo "<h1>Welcome to My Apache Server\!</h1>" | sudo tee /var/www/html/index.html

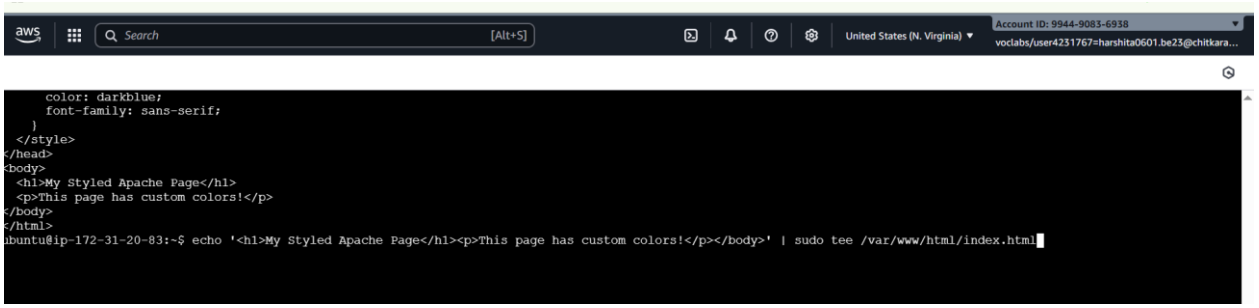

Figure 3.A.12

After running this command, refresh your browser page and you will see the changes

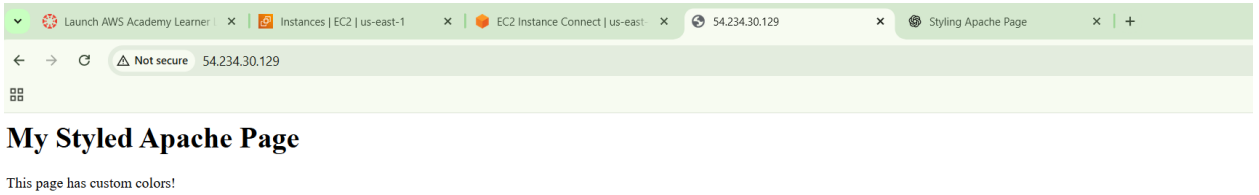

Figure 3.A.13

Step 6: For NGINX

1. Stop and disable Apache2 by running the below commands
   - sudo systemctl stop apache2
   - sudo systemctl disable apache2

   This will free up port 80

Figure 3.A.14

2. Install Nginx (if not already installed)
   - sudo apt update → refreshes the package list.

- sudo apt install nginx -y → installs Nginx and automatically confirms installation.

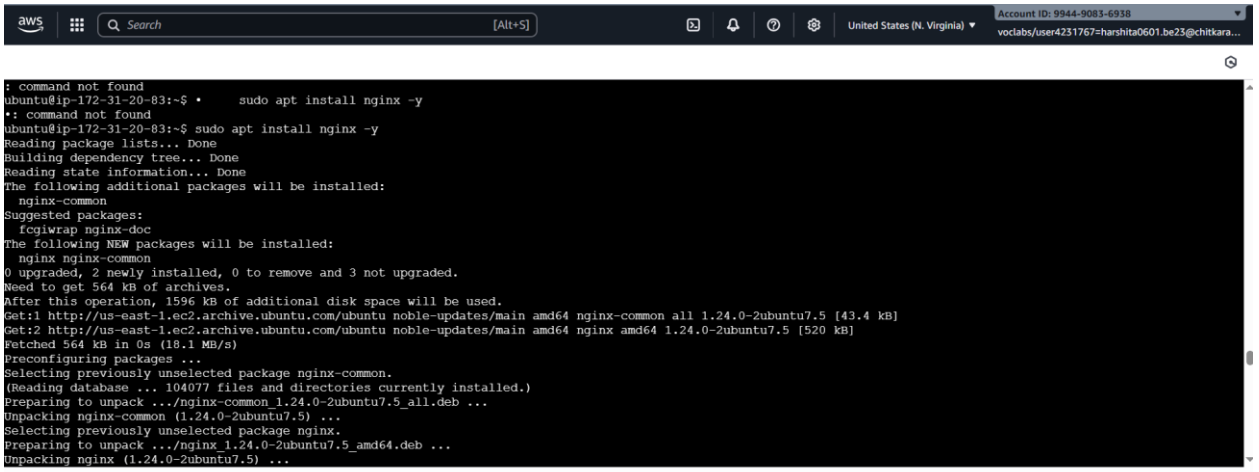

Figure 3.A.15

- sudo systemctl start nginx → Start Nginx
- sudo systemctl enable nginx → Ensures Nginx starts automatically when Ubuntu restarts.

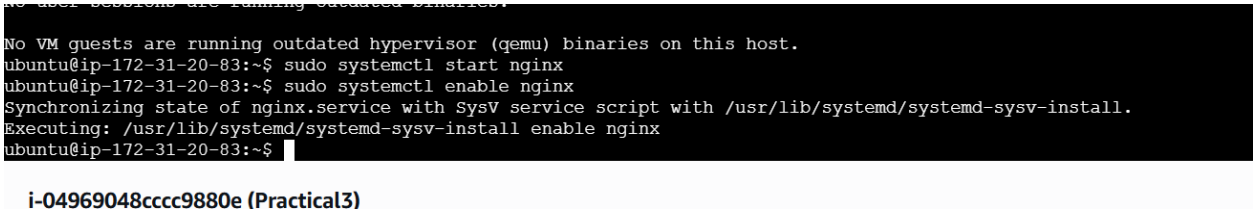

Figure 3.A.16

Step 7: Access Nginx in your browser

1. Find your Ubuntu server's public IP → hostname -I
2. Open a browser and visit → http://<Public_IP>
3. Make sure that nginx is using Port 80 by running the below command

- sudo nano /etc/nginx/sites-enabled/default

Figure 3.A.17

4. After running the above command restart the nginx → sudo systemctl restart nginx

Figure 3.A.18

Step 8: Customize Nginx Page

1. echo "<h1>Welcome to My Nginx Server!</h1>" | sudo tee /usr/share/nginx/html/index.html

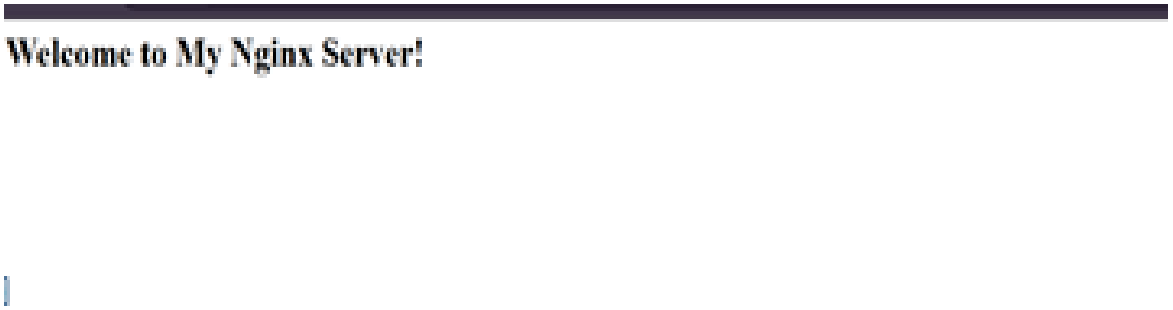

Figure 3.A.19

2. Create a Full HTML Page with Form and Style

```html
<!DOCTYPE html>
<html>
<head>
  <title>My Nginx Webpage</title>
  <style>
    body {
      background-color: #fdf6e3;
      font-family: Arial, sans-serif;
      text-align: center;
      padding: 50px;
    }
    input, textarea {
      padding: 10px;
      margin: 10px;
      width: 250px;
    }
    button {
      padding: 10px 20px;
      font-size: 16px;
    }
  </style>
</head>
<body>
  <h1>Welcome to My Nginx Page!</h1>
  <p>Please enter your name and message:</p>
  <form onsubmit="alert('Thank you for submitting!'); return false;">
    <input type="text" placeholder="Your Name" required><br>
    <textarea placeholder="Your Message" required></textarea><br>
    <button type="submit">Submit</button>
  </form>
  <p style="margin-top:40px;">This page is powered by Nginx on EC2!</p>
</body>
</html>
```

Figure 3.A.20

Figure 3.A.21

(B) Configure auto-scaling for EC2 instances to handle variable traffic.

Step 1: Create a Launch Template

1. Go to EC2 Console → Launch Template → Click Create Launch Template.

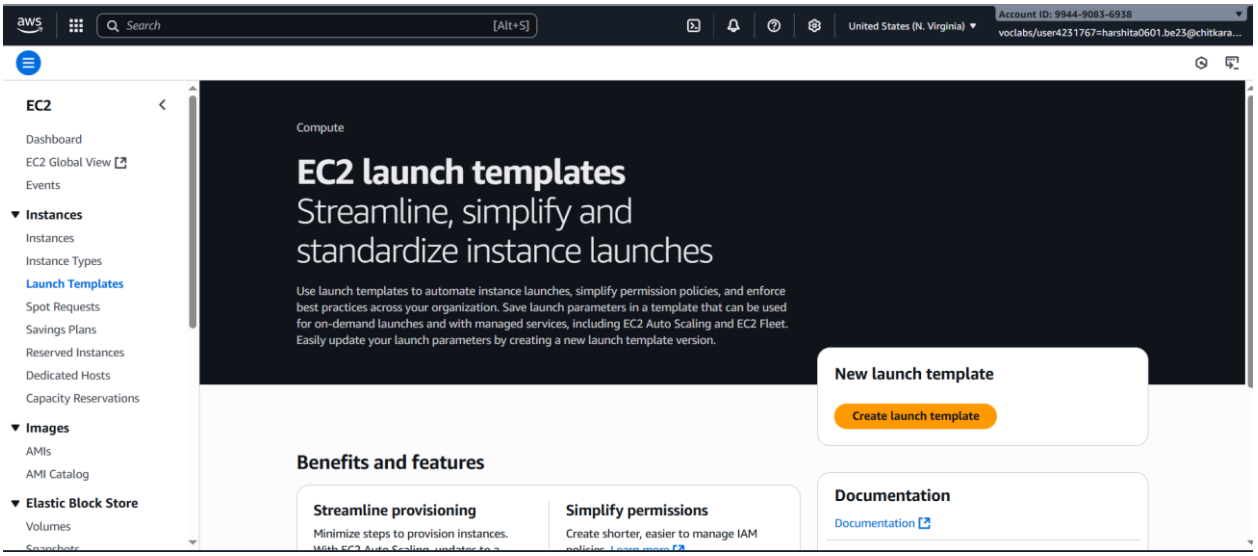

Figure 3.B.1

2. Fill in the basic information.
   - Name : eg:- web-server-template
   - Instance Type:- eg:- t2.micro or t3.micro
   - Key Pair:- Select your existing key.
   - Security Group:- Ensure it allows HTTP (port 80) or 8080.
   - User Data:- (Optional) Add shell script to auto-install web server if not in AMI.

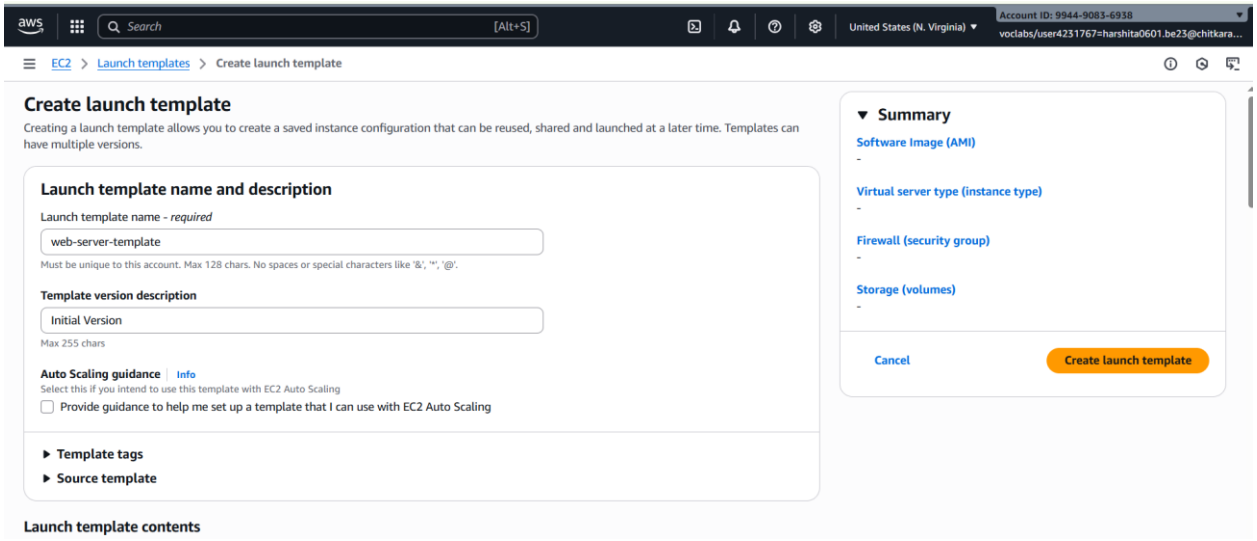

Figure 3.B.2

3. Click Create Launch Template.

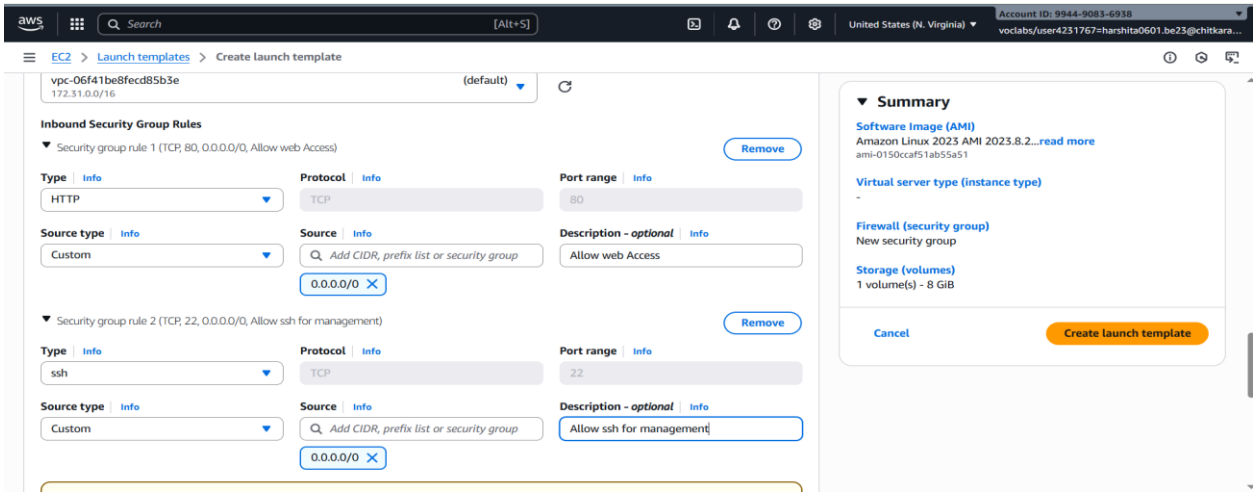

Figure 3.B.3

4. Successfully created

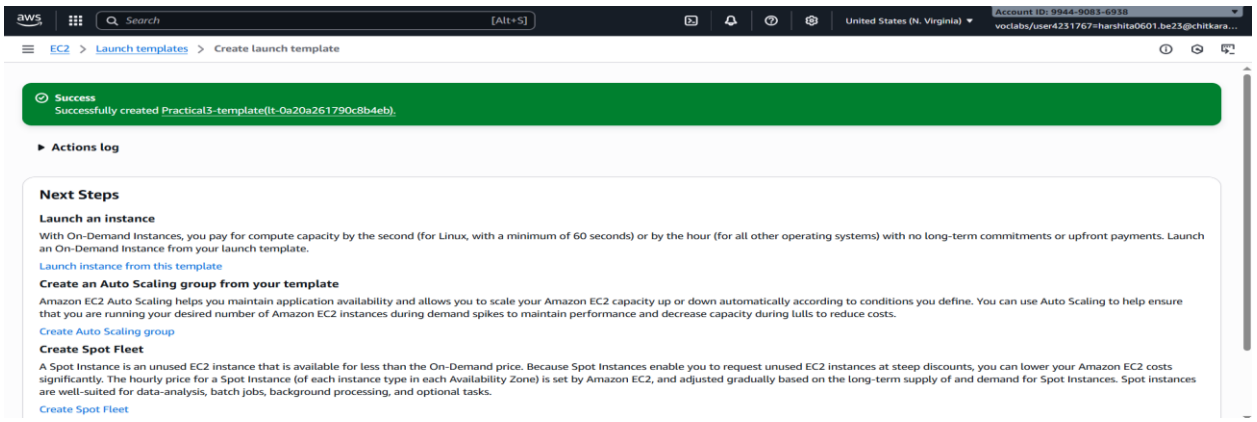

Figure 3.B.4

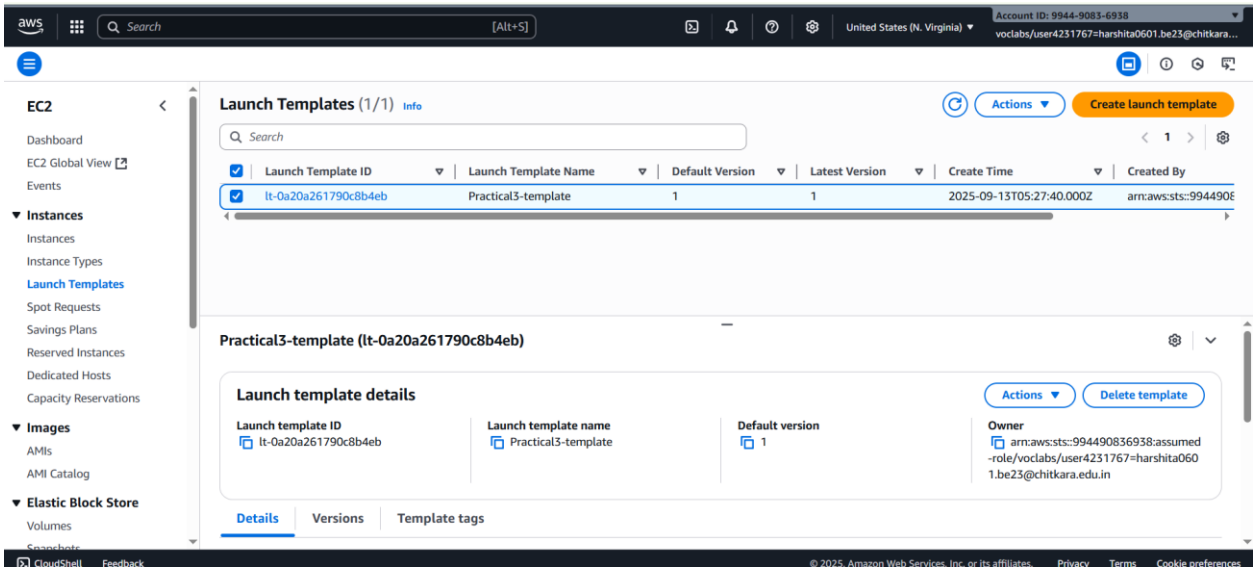

Figure 3.B.5

Step 2: Create an Auto Scaling Group (ASG)

1. Go to EC2 Console → Auto Scaling Groups → Click Create Auto Scaling Groups.

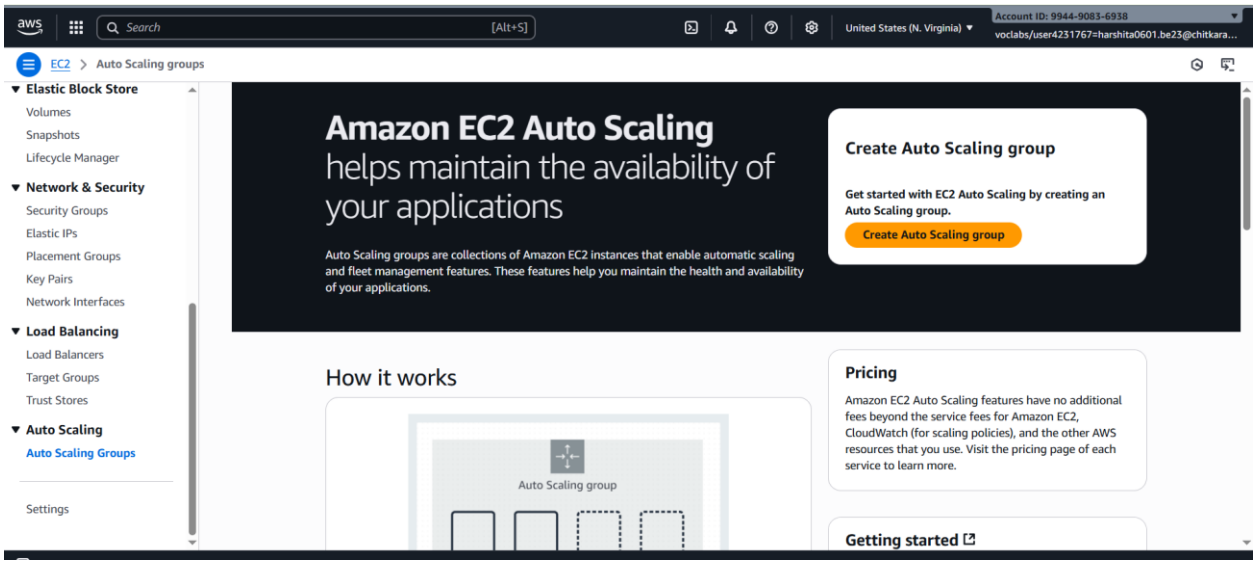

Figure 3.B.6

2. **Name:** e.g., web-auto-scaling
3. **Choose launch template:** Select the one you just created.
4. Click **Next**.

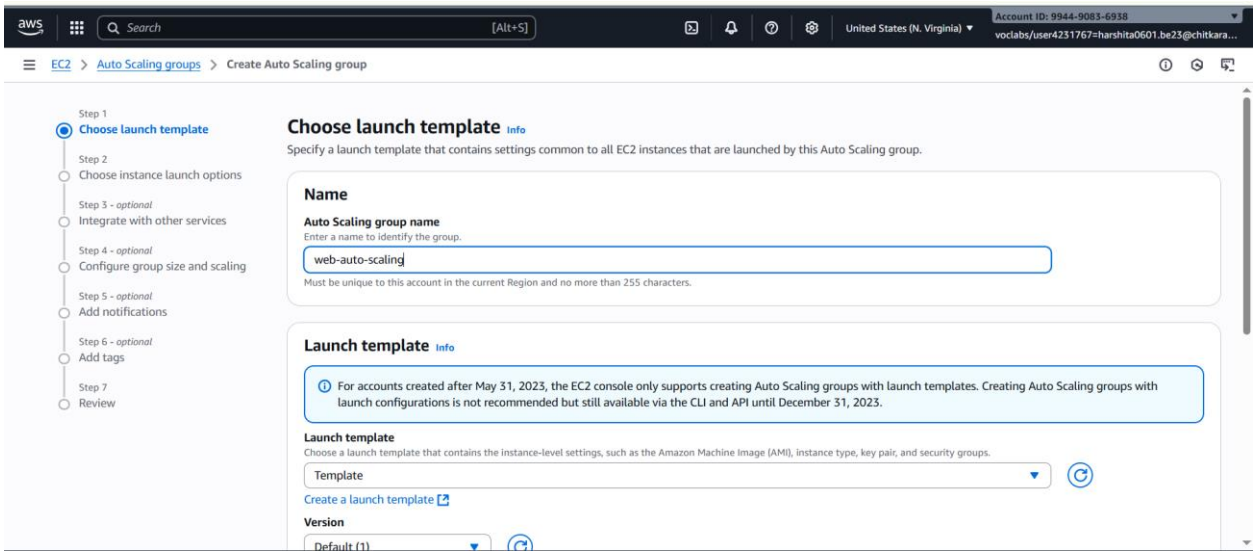

Figure 3.B.7

Step 3: Configure Network and Subnet

1. Select a **VPC** and **at least 2 subnets in different Availability Zones** (for high availability).
2. Enable **public IP** assignment if needed.

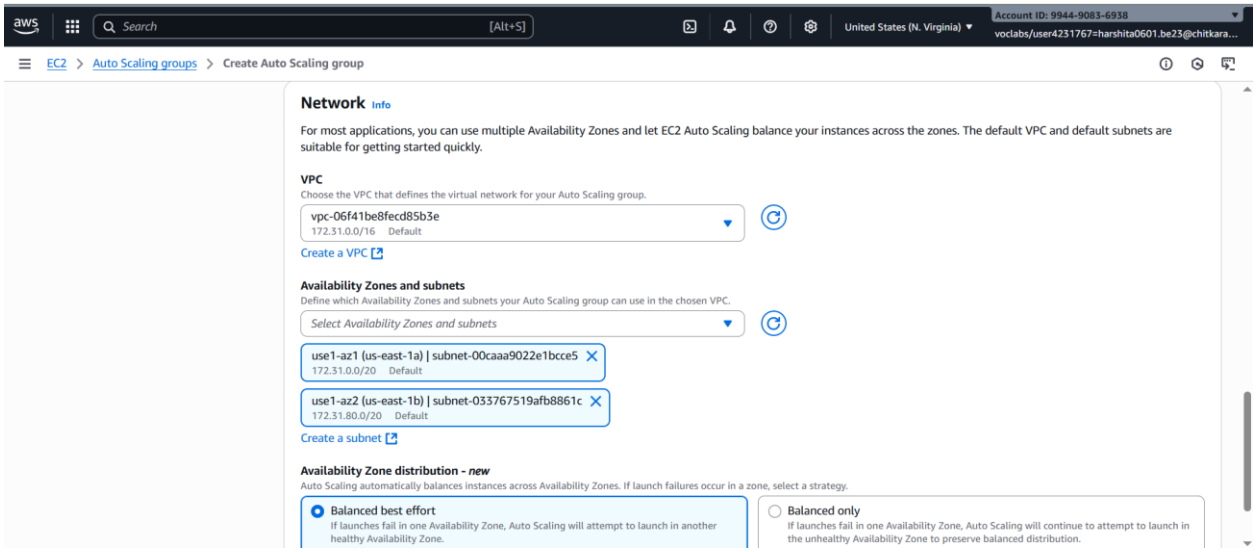

Figure 3.B.8

Step 4: Configure Load Balancer

1. Choose **Attach to a new load balancer** (Application Load Balancer).
2. Set:
    - Load balancer name
    - **Scheme:** Internet-facing
    - **Listeners:** HTTP port 80

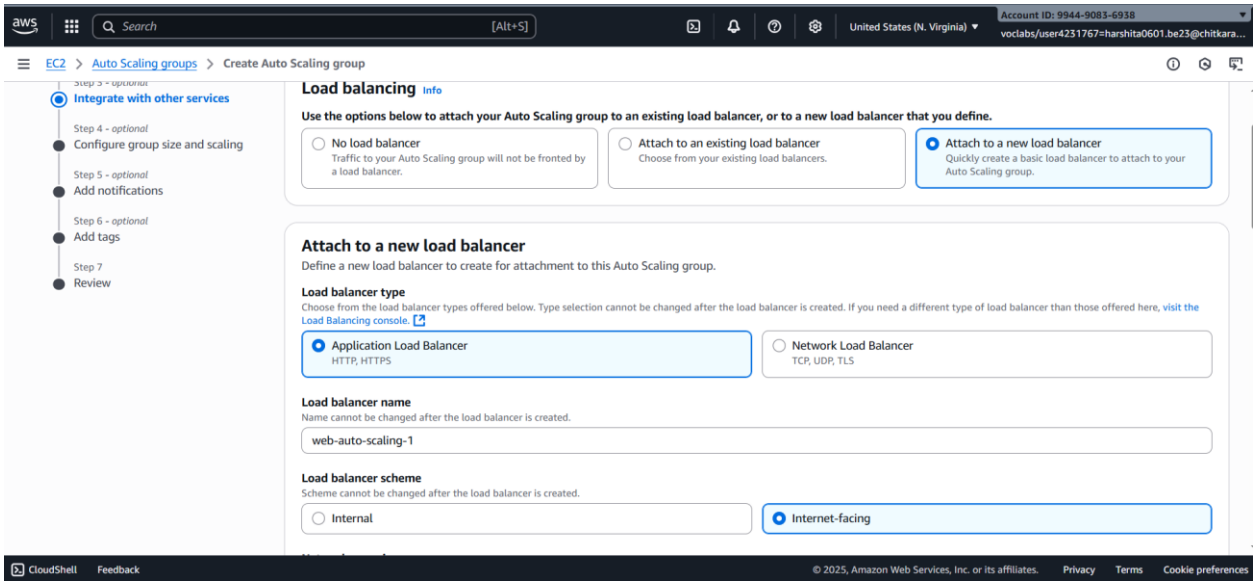

Figure 3.B.9

3. Add at least **2 Availability Zones**.
4. Create or select a **Target Group**, protocol: HTTP, port 80 (or 8080 if you're using it).

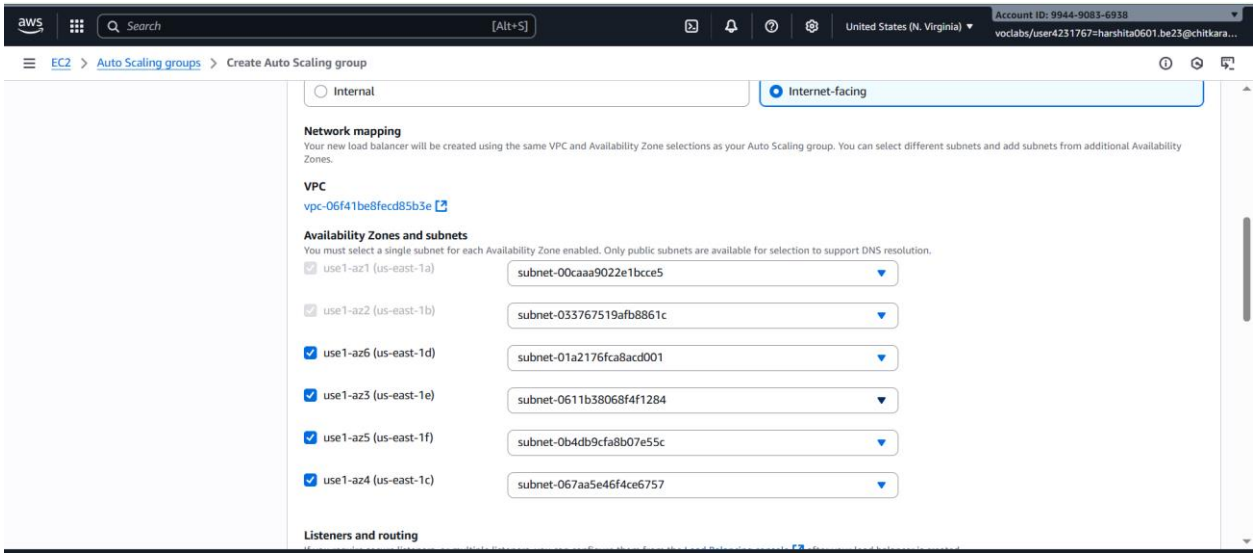

Figure 3.B.10

Step 5: Configure Scaling Policies

1. **Desired capacity:** 2 (minimum recommended)
2. Minimum capacity: 1
3. **Maximum capacity:** 3 (or higher based on expected traffic)

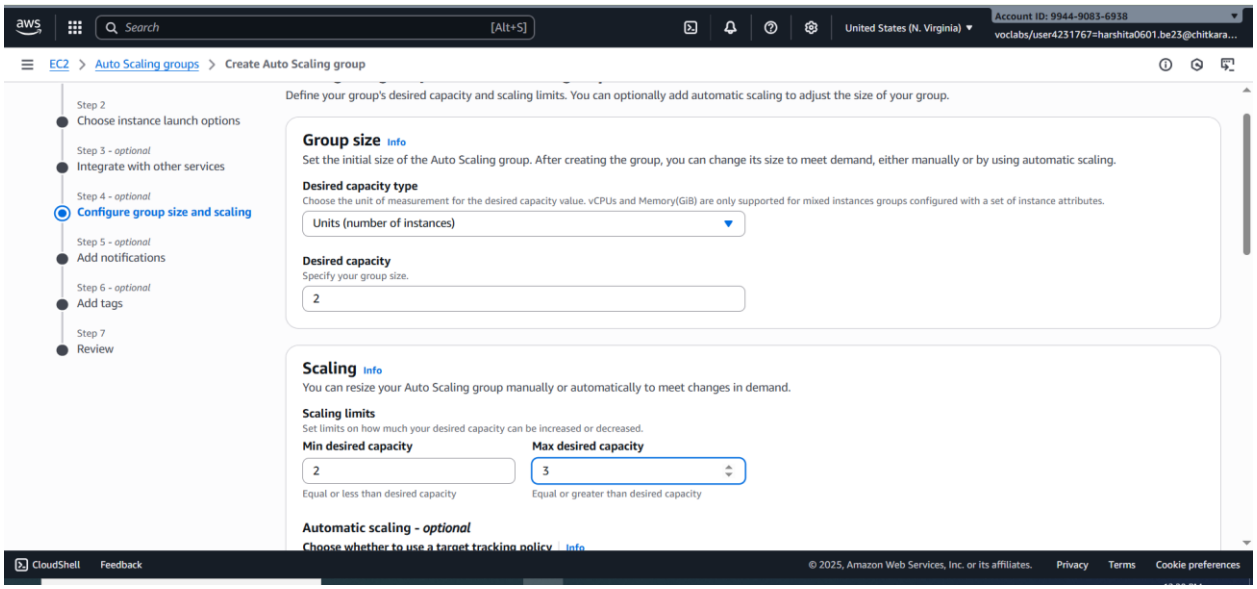

Figure 3.B.11

4. Choose **Target tracking scaling policy**:
    - Metric type: **Average CPU utilization**
    - Target value: e.g., **60%**
    - This means: add or remove instances to keep average CPU at 60%.

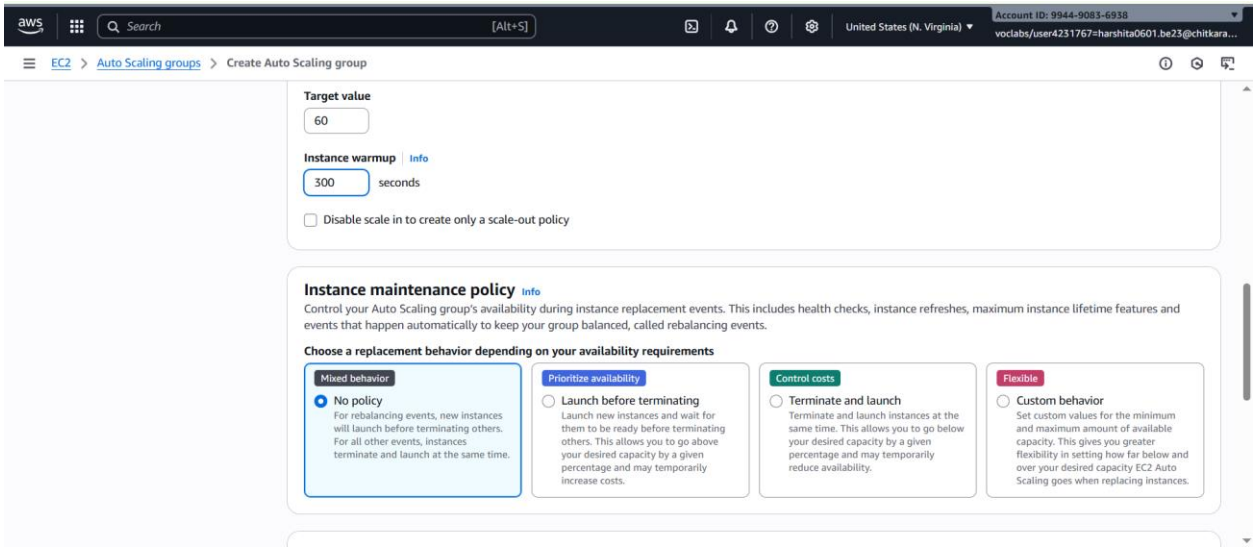

Figure 3.B.12

Step 6: Review and Create

- Review all configuration.
- Click **Create Auto Scaling group**.

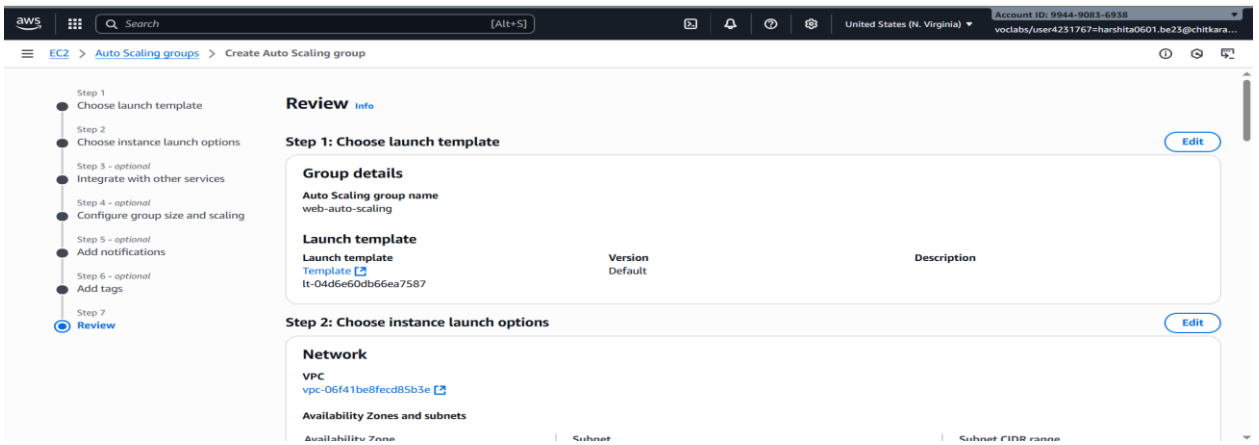

Figure 3.B.13

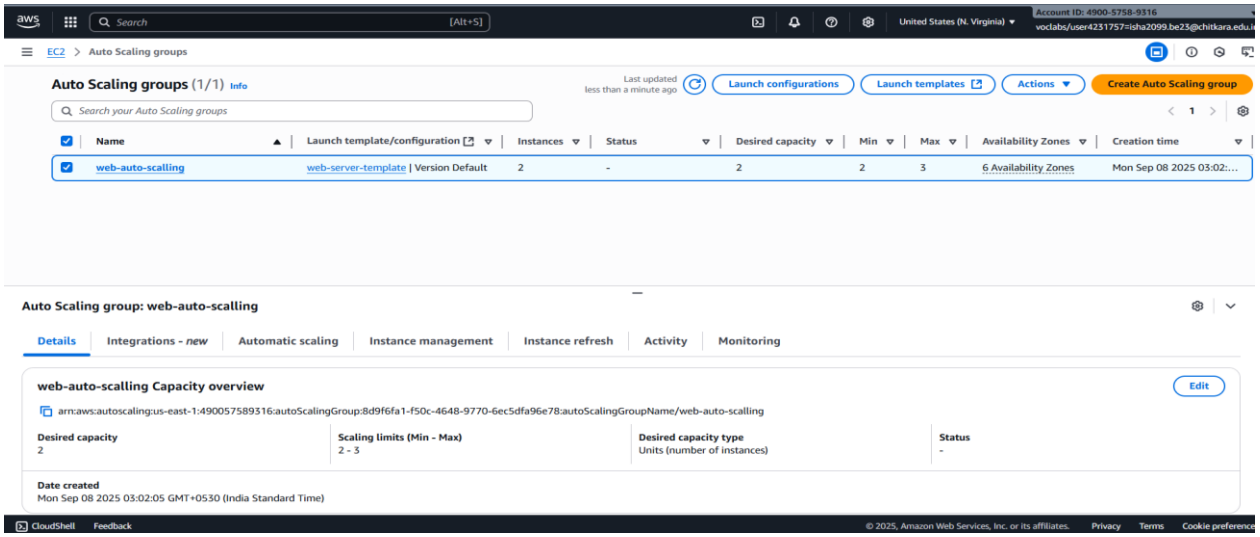

Figure 3.B.14

Step 7: Check the instances

You will see at least two instances created with the help of Auto Scaling.

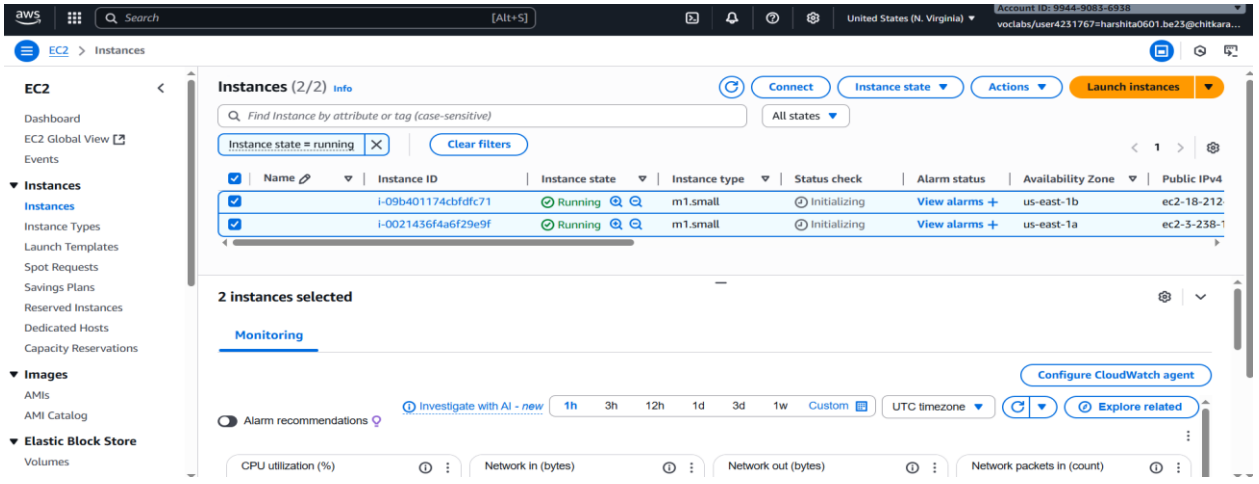

Figure 3.B.15

Practical No. 4

Practical Title: Create Amazon S3 Bucket, Upload Objects, Configure Access, and Host Static Website.

Objective:

1. To create an Amazon S3 bucket for storing data.
2. To upload objects (files such as HTML, CSS, images) into the S3 bucket.
3. To configure access controls using bucket policies for secure and controlled access.
4. To enable static website hosting on the S3 bucket.

Step-by-Step Procedure with Screenshots:

Step1: Create S3 Bucket

1. Sign in to your AWS Management Console → Go to S3 service.

Figure 4.1

2. Click on Create bucket.

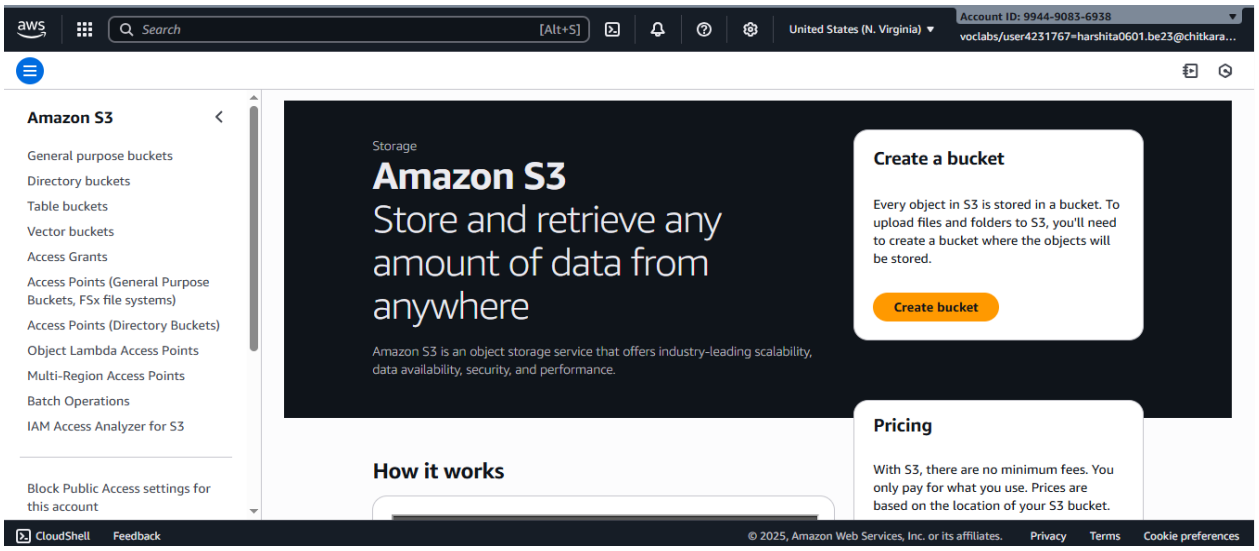

Figure 4.2

3. Enter a **unique bucket name** (e.g., my-static-site-bucket).
    - Bucket names must be globally unique.

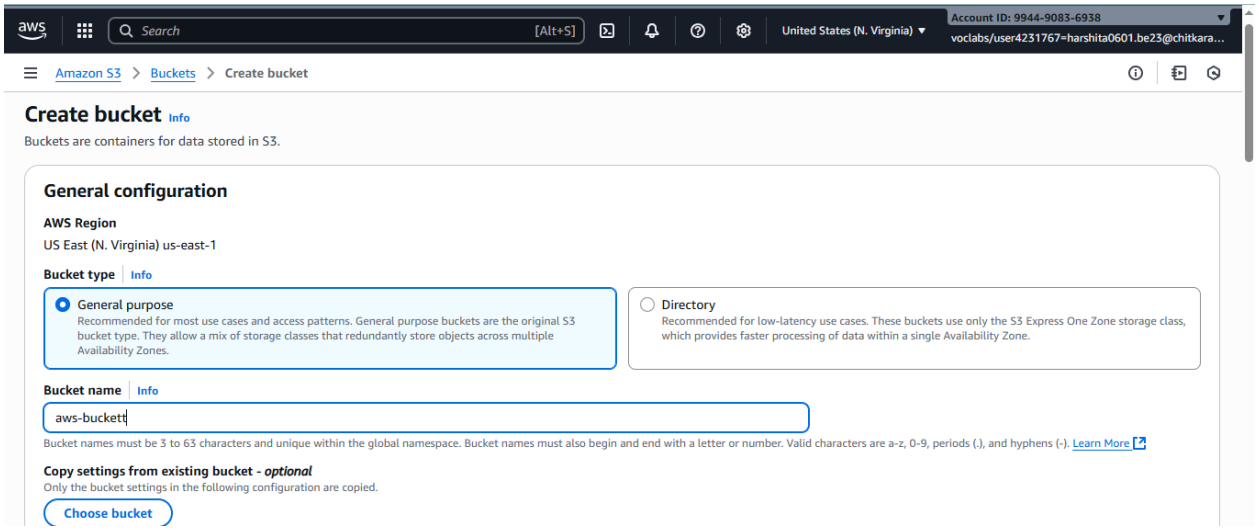

Figure 4.3

4. Select an **AWS Region** (pick the one closest to your users).
5. In **Block Public Access settings**, **uncheck "Block all public access"** → Confirm the warning (since you need the bucket public for hosting).

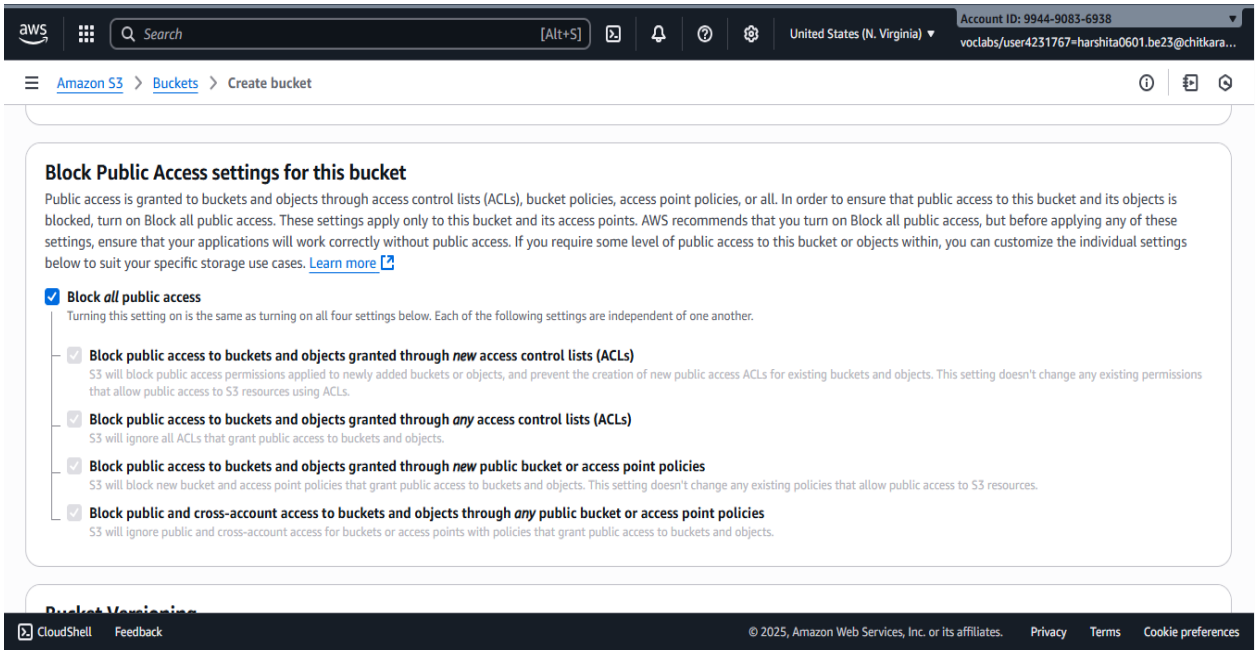

Figure 4.4

6. Click **Create bucket**.

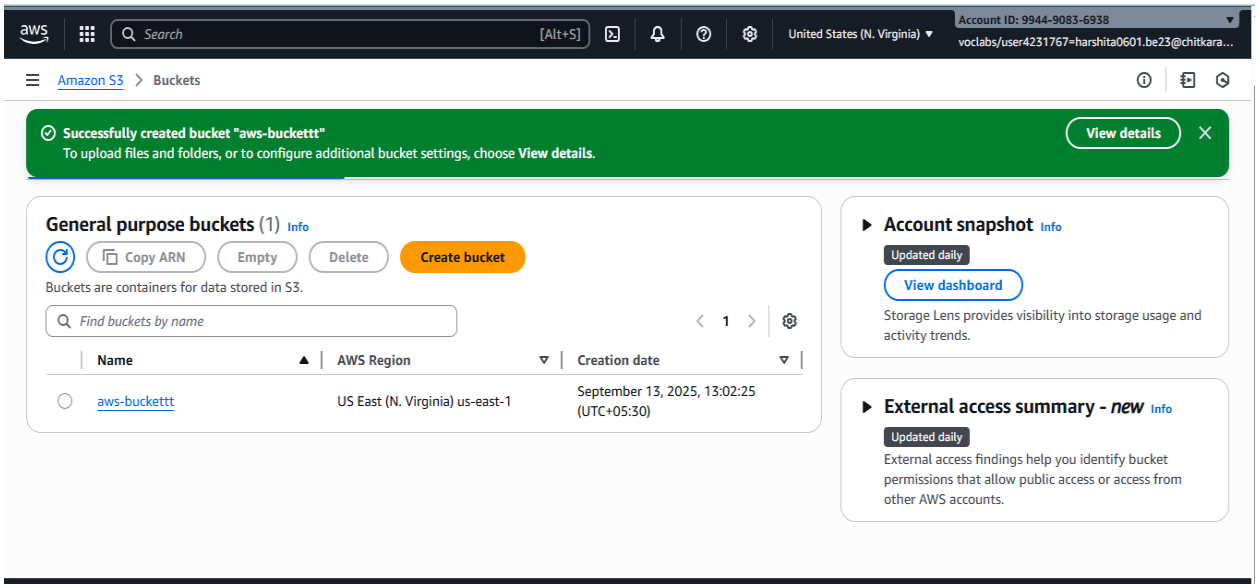

Figure 4.5

Step2: Upload Objects

1. Open your bucket.

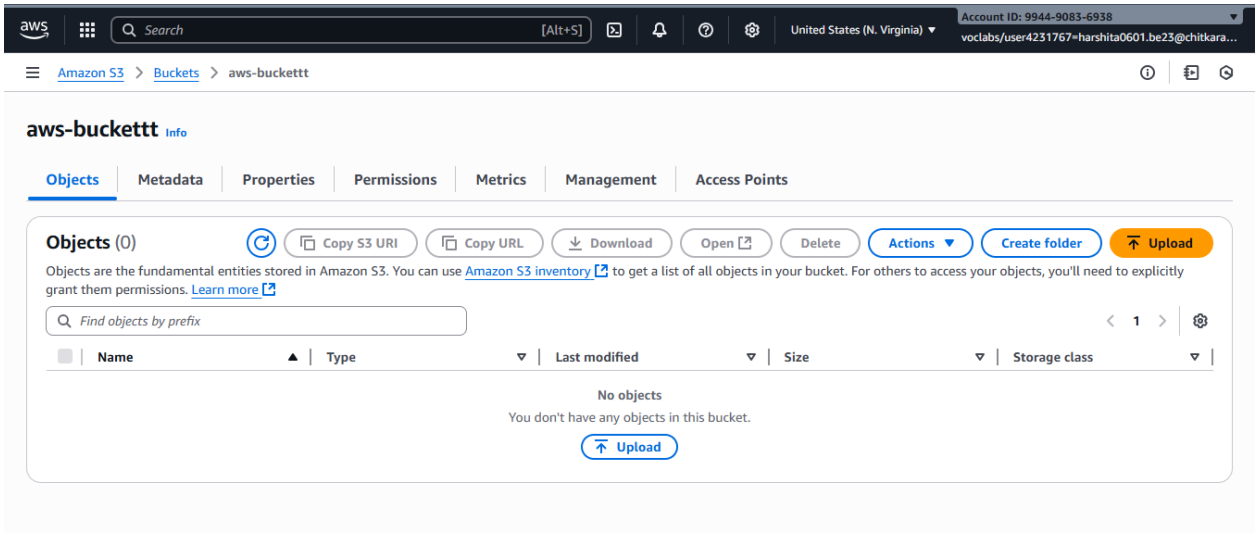

Figure 4.6

2. Select your files or pdf for uploading.

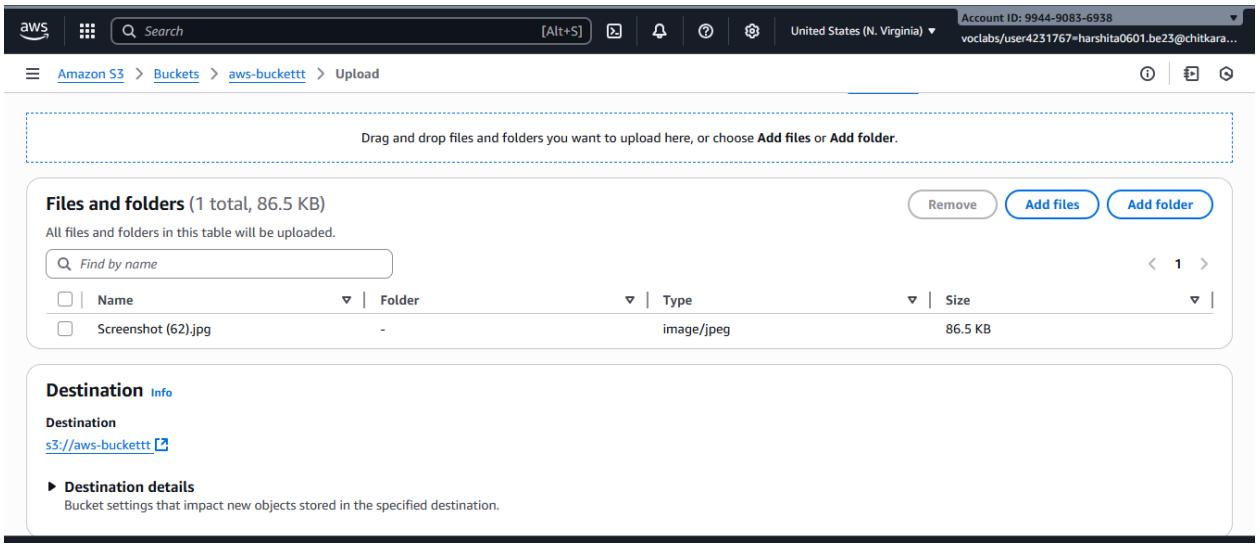

Figure 4.7

3. Click Upload.

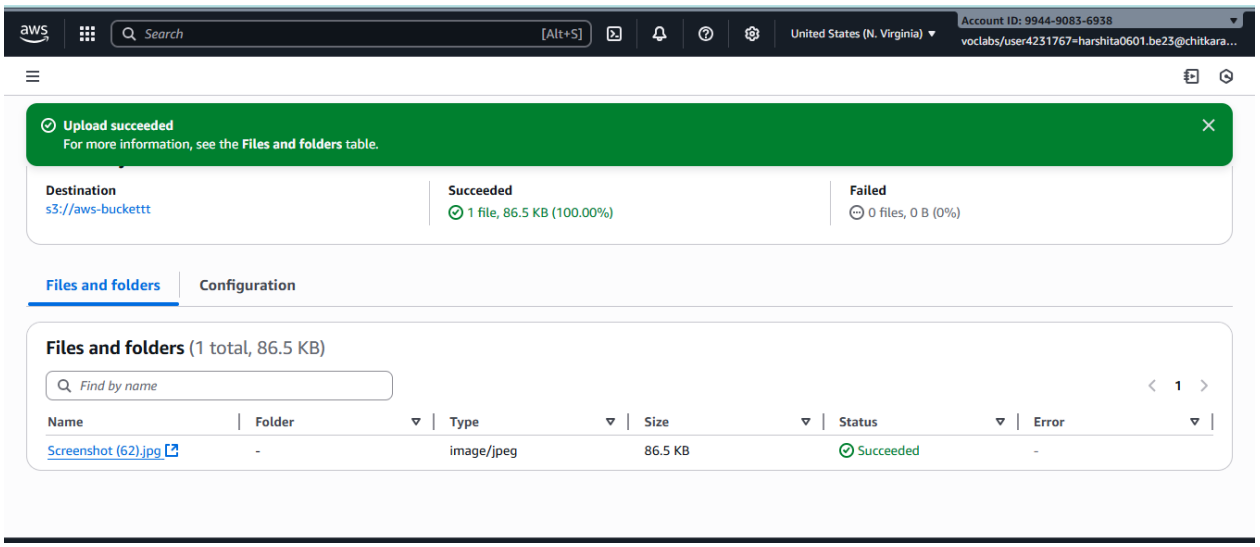

Figure 4.8

Step 3: Set Permissions with a Bucket Policy

Since you want users to access your files, you need to set a **bucket policy**.

1. Open your bucket → **Permissions** tab → **Bucket Policy** → **Policy Generator**.

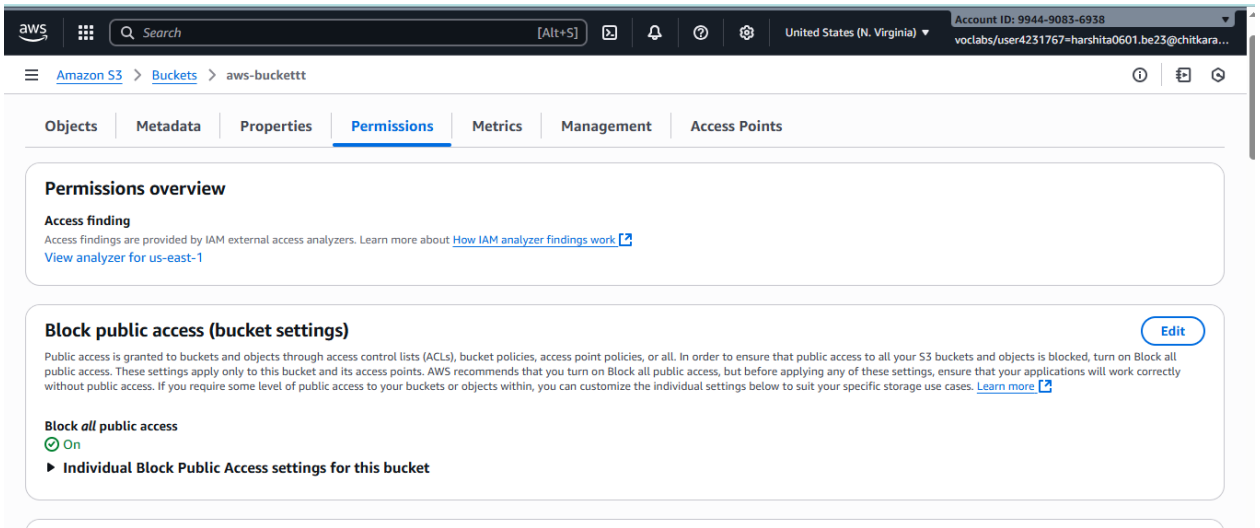

Figure 4.9

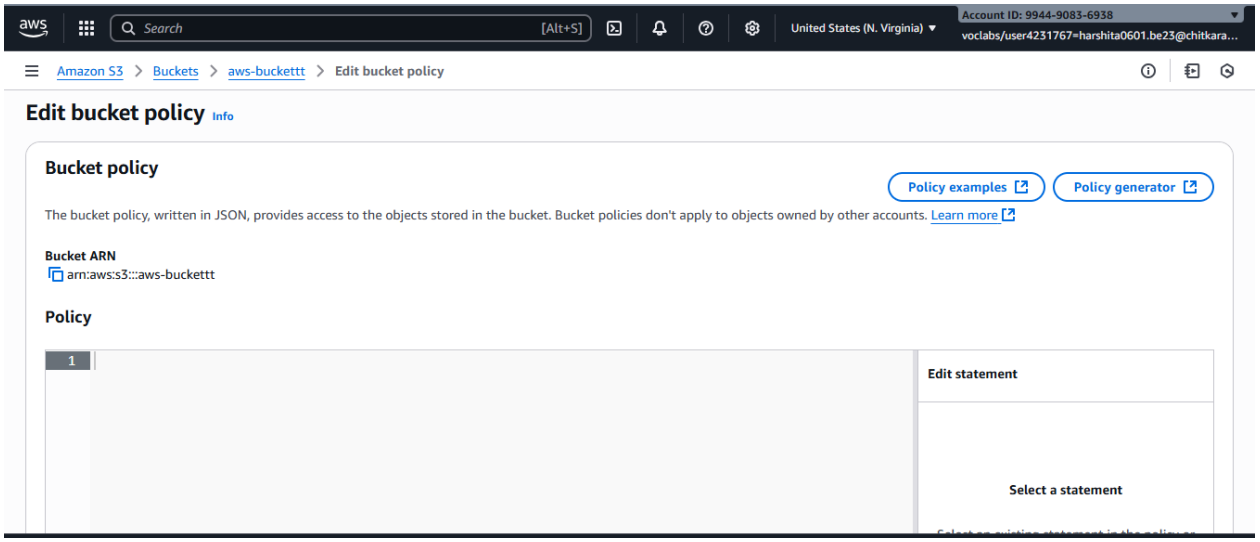

Figure 4.10

2. **Select:**
   - **Policy Type:** S3 Bucket Policy
   - **Effect:** Allow

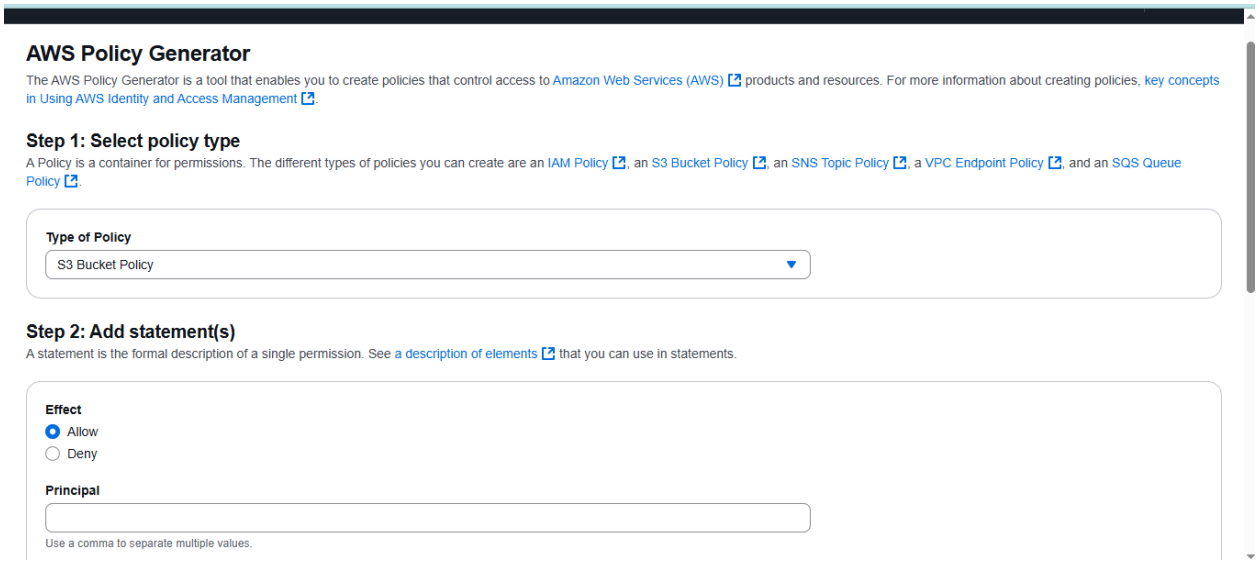

Figure 4.11

- **Principal:** * (means everyone)
- **Actions:** choose → GetObject
- **ARN:** all resources

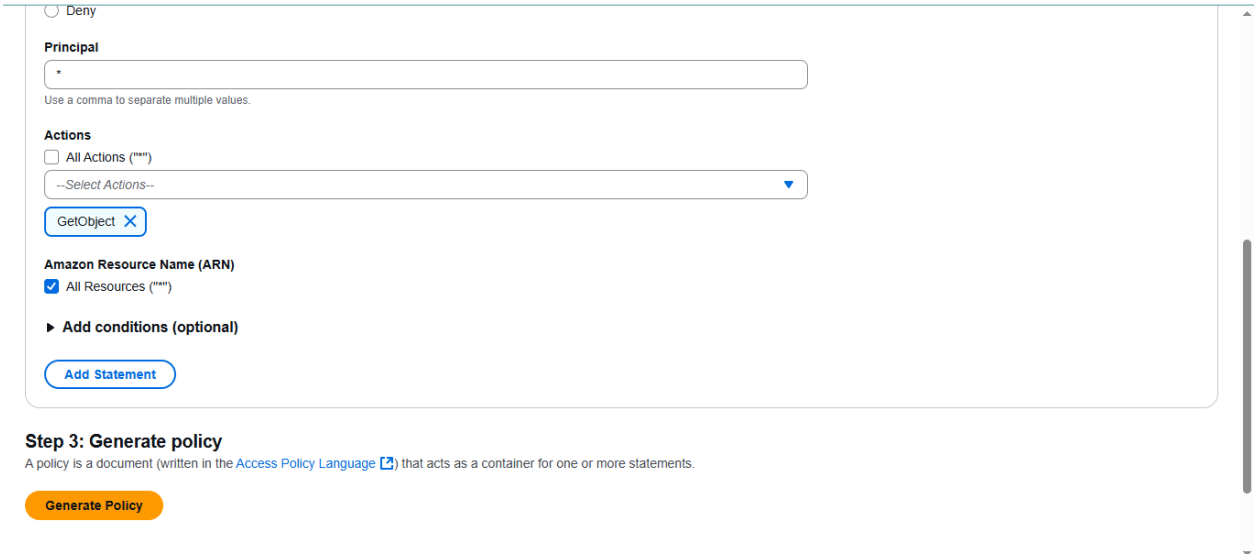

Figure 4.12

3. Click **Add Statement** → Then **Generate Policy**.

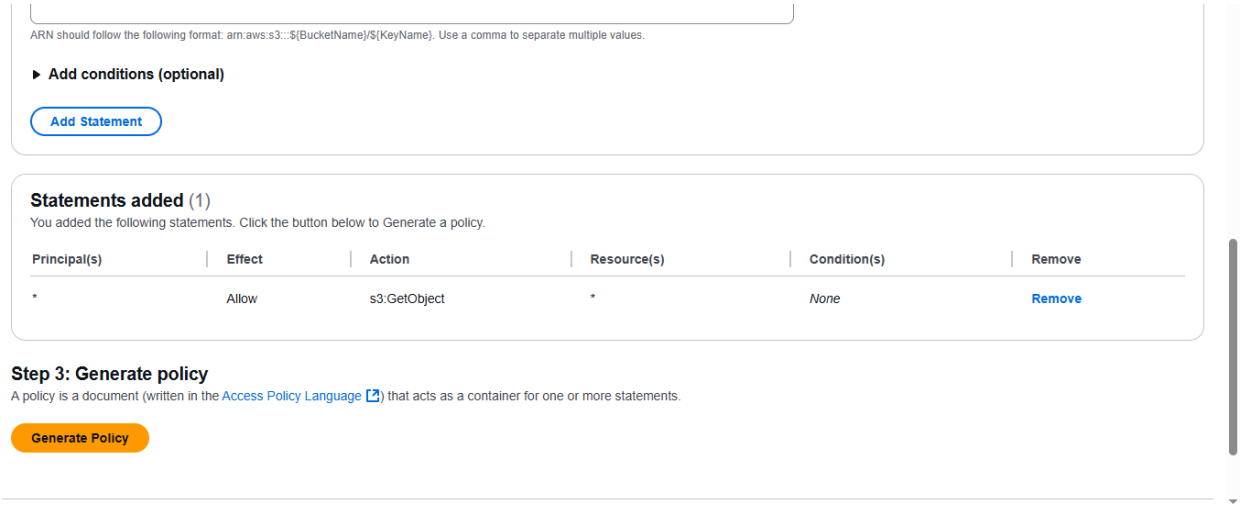

Figure 4.13

4. Copy the generated JSON and paste it into your **Bucket Policy editor**.

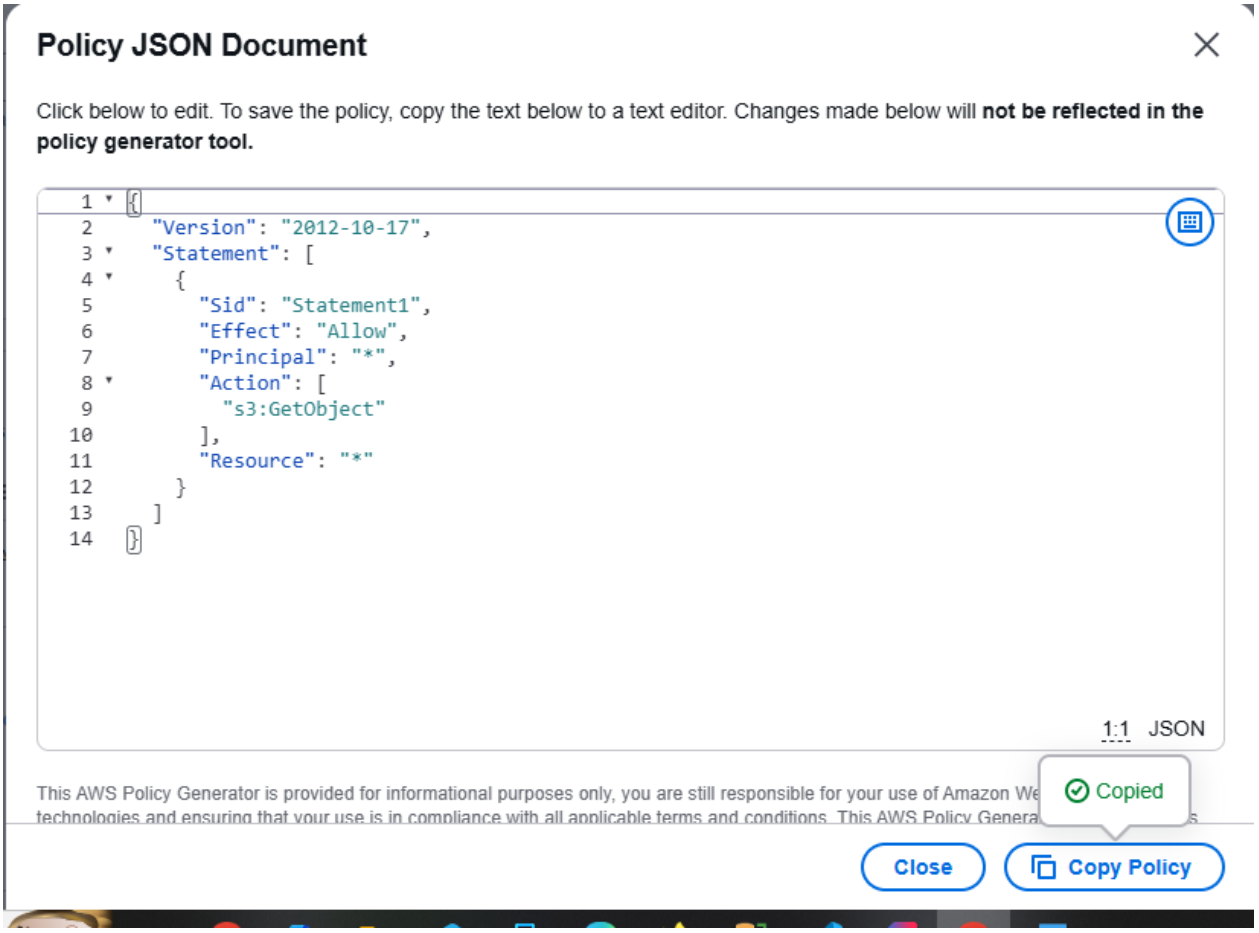

Figure 4.14

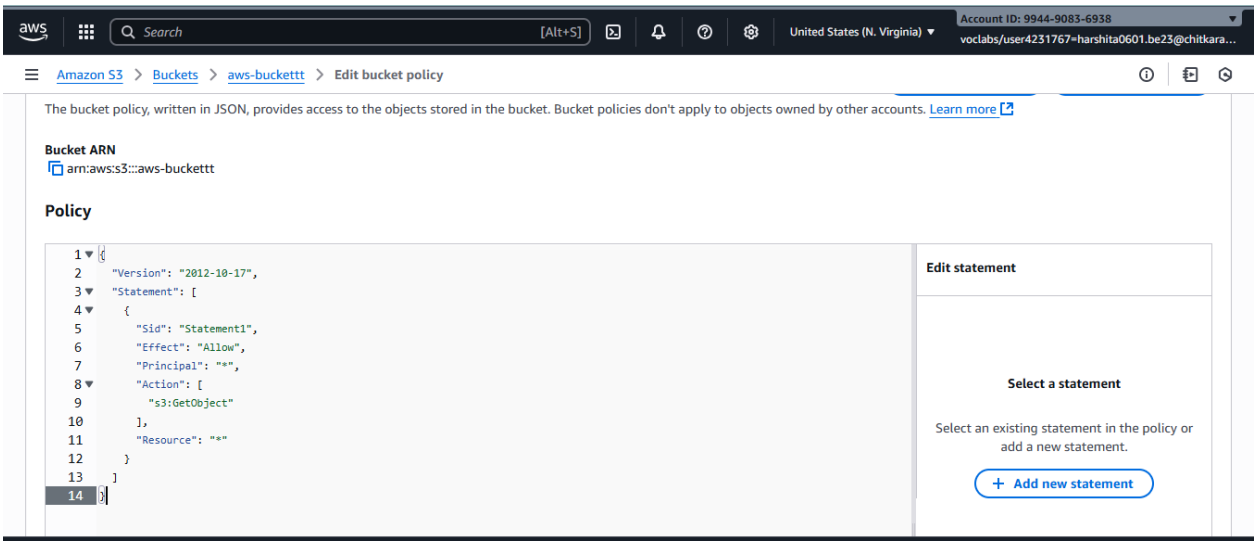

Figure 4.15

5. Save the policy :- this allows public read access to all the objects in your bucket.

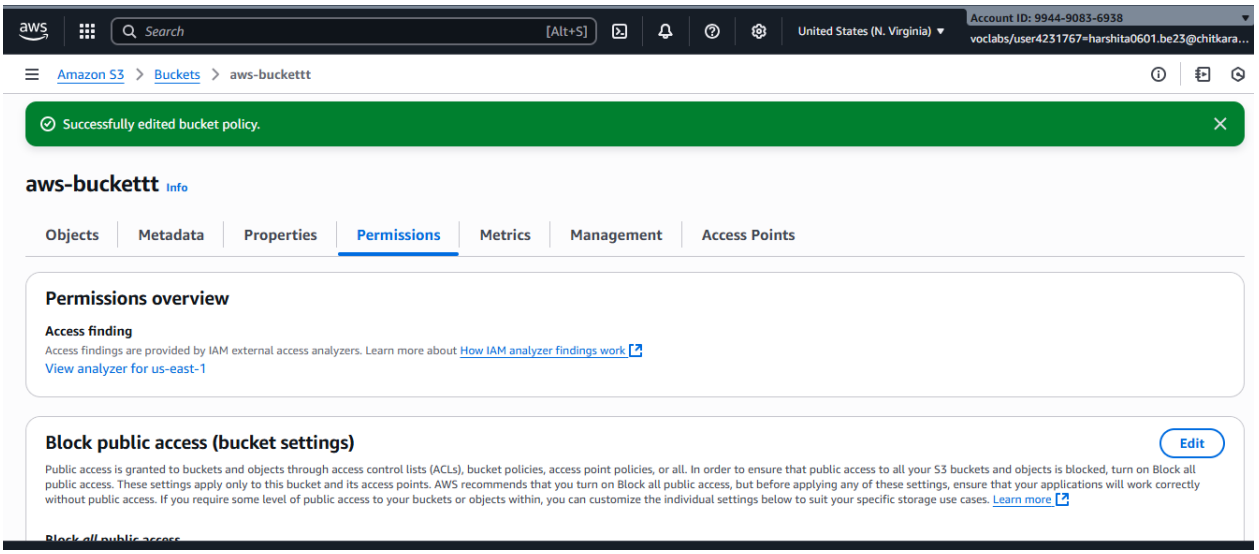

Figure 4.16

Step 4: Enable Static Website Hosting

1. Go to your bucket → **Properties tab**.

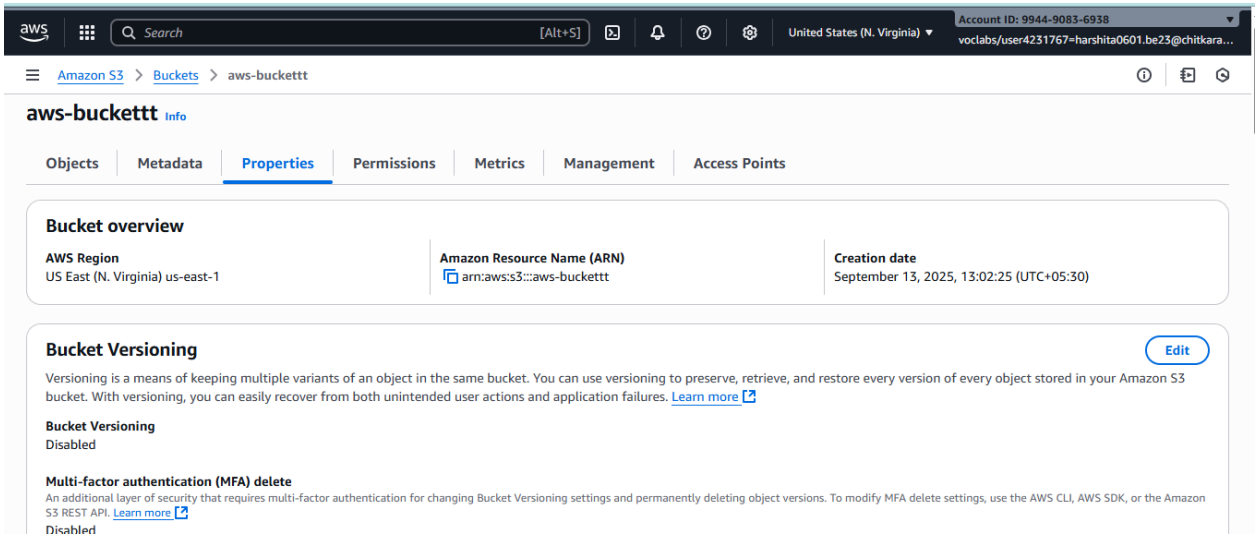

Figure 4.17

2. Scroll to **Static website hosting** → Click **Edit**.
3. Select **Enable**.
4. For **Index document**, type: index.html.
5. Save Changes.

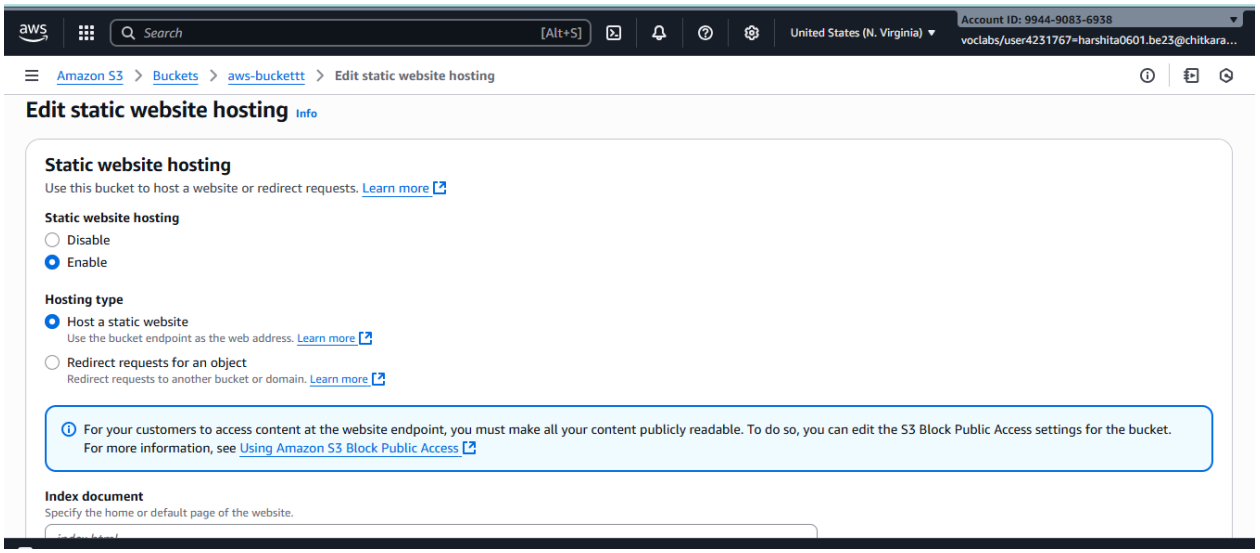

Figure 4.18

Step 5: Access Your Website

- **After enabling hosting, AWS gives you a bucket website endpoint URL, something like:**

  http://my-static-site-bucket.s3-website-us-east-1.amazonaws.com

- **Share this URL → Your static website is live!**

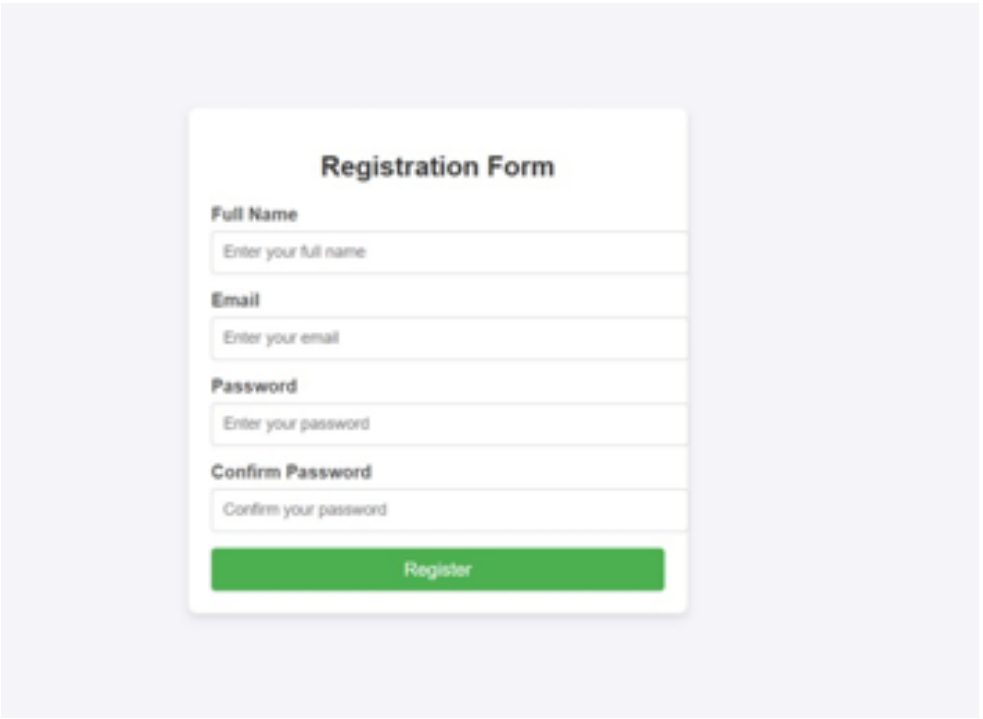

Figure 4.19

Learning Outcomes:

1. Created an Amazon S3 bucket and understood its role in cloud storage.
2. Uploaded and managed objects within an S3 bucket.
3. Configured bucket policies and access controls to secure data and manage permissions.
4. Enabled and tested static website hosting using Amazon S3.
5. Understood how S3 can be used for cost-effective, scalable, and highly available content delivery